

```
# IMPORTANT: RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES,
# THEN FEEL FREE TO DELETE THIS CELL.
# NOTE: THIS NOTEBOOK ENVIRONMENT DIFFERS FROM KAGGLE'S PYTHON
# ENVIRONMENT SO THERE MAY BE MISSING LIBRARIES USED BY YOUR
# NOTEBOOK.
import kagglehub
manishkr1754_cardekho_used_car_data_path = kagglehub.dataset_download('manishkr1754/cardekho-used-car-data')
mohitkumar282_used_car_dataset_path = kagglehub.dataset_download('mohitkumar282/used-car-dataset')

print('Data source import complete.')
```

```
# This Python 3 environment comes with many helpful analytics libraries installed
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) that gets preserved as output when you create a new notebook.
# You can also write temporary files to /kaggle/temp/, but they won't be saved outside of the current session

# Use the kagglehub client library to attach Kaggle resources like competitions, datasets, and models to your session.
# Learn more about kagglehub: https://github.com/Kaggle/kagglehub/blob/main/README.md

import kagglehub
# kagglehub.dataset_download('<owner>/<dataset-slug>')
```

```
/kaggle/input/datasets/mohitkumar282/used-car-dataset/used_cars_dataset_v2.csv
/kaggle/input/datasets/mohitkumar282/used-car-dataset/used_car_dataset.csv
/kaggle/input/datasets/manishkr1754/cardekho-used-car-data/cardekho_dataset.csv
```

```
import os

for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))
```

```
/kaggle/input/datasets/mohitkumar282/used-car-dataset/used_cars_dataset_v2.csv
/kaggle/input/datasets/mohitkumar282/used-car-dataset/used_car_dataset.csv
/kaggle/input/datasets/manishkr1754/cardekho-used-car-data/cardekho_dataset.csv
```

1. Load the Dataset

The dataset is provided by Kaggle as a CSV file.

We will use Pandas to read the CSV file into a DataFrame.

A DataFrame is a two-dimensional table containing rows and columns. It allows us to inspect, clean, analyze, and prepare the dataset for machine learning.

```
import pandas as pd

dataset_path = "/kaggle/input/datasets/manishkr1754/cardekho-used-car-data/cardekho_dataset.csv"

df = pd.read_csv(dataset_path)

print("Dataset loaded successfully.")
```

```
Dataset loaded successfully.
```

```
print(df.shape)
```

```
(15411, 14)
```

2. Understand the Dataset

Before training a machine learning model, we need to understand what information is available in the dataset.

We will inspect:

- column names
- data types
- sample records
- missing values
- duplicate records
- numerical statistics

This process is called Exploratory Data Analysis (EDA).

```
print(df.columns)
```

```
Index(['Unnamed: 0', 'car_name', 'brand', 'model', 'vehicle_age', 'km_driven',
      'seller_type', 'fuel_type', 'transmission_type', 'mileage', 'engine',
      'max_power', 'seats', 'selling_price'],
      dtype='object')
```

```
df.head()
```

	Unnamed: 0	car_name	brand	model	vehicle_age	km_driven	seller_type	fuel_type	transmission_type	mileage
0	0	Maruti Alto	Maruti	Alto	9	120000	Individual	Petrol	Manual	19.70
1	1	Hyundai Grand	Hyundai	Grand	5	20000	Individual	Petrol	Manual	18.90
2	2	Hyundai i20	Hyundai	i20	11	60000	Individual	Petrol	Manual	17.00

3. Inspect Data Types

Different columns contain different types of information.

Numerical columns can usually be used directly by machine-learning algorithms, while categorical/text columns need to be converted into numerical representations before training.

We will inspect the data types before deciding how to preprocess the data.

```
df.dtypes
```

```
Unnamed: 0      int64
car_name        object
brand           object
model           object
vehicle_age     int64
km_driven       int64
seller_type     object
fuel_type       object
transmission_type object
mileage         float64
engine          int64
max_power       float64
seats           int64
selling_price   int64
dtype: object
```

4. Check Missing Values

Machine-learning models generally cannot work directly with missing values in the way raw datasets contain them.

We will identify which columns contain missing values before deciding how to handle them.

```
df.isnull().sum()
```

```
Unnamed: 0      0
car_name        0
brand           0
model           0
vehicle_age     0
```

```

km_driven      0
seller_type    0
fuel_type      0
transmission_type 0
mileage        0
engine         0
max_power      0
seats          0
selling_price  0
dtype: int64

```

5. Check Duplicate Records

Duplicate rows can cause the same car record to appear more than once.

We will count duplicate rows before training the model.

```
df.duplicated().sum()
```

```
np.int64(0)
```

6. Numerical Statistics

We will summarize the numerical columns to understand their typical values and ranges.

This helps us detect unusual values and understand the scale of the data before machine learning.

```
df.describe()
```

	Unnamed: 0	vehicle_age	km_driven	mileage	engine	max_power	seats	selling_price
count	15411.000000	15411.000000	1.541100e+04	15411.000000	15411.000000	15411.000000	15411.000000	1.541100e+04
mean	9811.857699	6.036338	5.561648e+04	19.701151	1486.057751	100.588254	5.325482	7.749711e+05
std	5643.418542	3.013291	5.161855e+04	4.171265	521.106696	42.972979	0.807628	8.941284e+05
min	0.000000	0.000000	1.000000e+02	4.000000	793.000000	38.400000	0.000000	4.000000e+04
25%	4906.500000	4.000000	3.000000e+04	17.000000	1197.000000	74.000000	5.000000	3.850000e+05
50%	9872.000000	6.000000	5.000000e+04	19.670000	1248.000000	88.500000	5.000000	5.560000e+05
75%	14668.500000	8.000000	7.000000e+04	22.700000	1582.000000	117.300000	5.000000	8.250000e+05
max	19543.000000	29.000000	3.800000e+06	33.540000	6592.000000	626.000000	9.000000	3.950000e+07

7. Understand Categorical Feature Cardinality

Not every categorical column should necessarily be encoded in exactly the same way.

Before choosing an encoding strategy, we need to understand how many different categories each categorical feature contains.

For example, `fuel_type` usually has relatively few categories, while `model` may contain many different categories.

We will inspect the categorical features using:

- `value_counts()` to see how frequently each category appears.
- `nunique()` to count how many different categories exist.

What does `nunique()` mean?

`nunique()` means **number of unique values**.

For example, if a column contains:

Petrol

Petrol

Diesel

Petrol

CNG

there are 3 unique values:

- Petrol
- Diesel
- CNG

Therefore:

```
nunique()
```

```
df["fuel_type"].value_counts()
```

```
fuel_type
Petrol      7643
Diesel      7419
CNG         301
LPG          44
Electric      4
Name: count, dtype: int64
```

```
df["brand"].nunique()
```

```
32
```

```
df["model"].nunique()
```

```
120
```

```
df["fuel_type"].nunique()
```

```
5
```

8. Inspect Categorical Feature Distributions

We have identified that categorical features can contain different numbers of categories.

Before choosing an encoding strategy, we will inspect the distribution of each categorical feature.

`value_counts()` shows how frequently each category occurs.

This is useful because a category that appears only a few times may behave differently from a category with thousands of examples.

We will inspect:

- brand
- model
- seller_type
- fuel_type
- transmission_type

```
categorical_columns = [
    "brand",
    "model",
    "seller_type",
    "fuel_type",
    "transmission_type"
]

for column in categorical_columns:
    print(f"\n==== {column} =====")
    print(df[column].value_counts())
```

```
==== brand =====
brand
Maruti      4992
Hyundai     2982
Honda       1485
Mahindra    1011
Toyota       793
Ford         790
Volkswagen   620
Renault      536
BMW          439
Tata         430
Mercedes-Benz 337
Skoda        334
Audi         192
Datsun       170
Jaguar        59
Land Rover   51
Jeep          41
Kia           32
Porsche       21
Volvo         20
```

```

MG                19
Mini              17
Nissan            11
Lexus             10
Isuzu             8
Bentley           3
Maserati          2
ISUZU             2
Ferrari           1
Mercedes-AMG      1
Rolls-Royce       1
Force             1
Name: count, dtype: int64

===== model =====
model
i20                906
Swift Dzire        890
Swift              781
Alto               778
City              757
...
Altroz             1
C                  1
Ghost              1
Quattroporte       1
Gurkha             1
Name: count, Length: 120, dtype: int64

===== seller_type =====
seller_type
Dealer            9539
Individual         5699
Trustmark Dealer   173
Name: count, dtype: int64

```

9. Understand the Target Variable

Our goal is to predict `selling_price`.

This is the target variable, also called the dependent variable.

Before training the model, we need to understand how the target values are distributed.

We will inspect:

- minimum selling price
- maximum selling price
- average selling price
- median selling price
- distribution of prices

Understanding the target helps us identify unusual values and understand the problem we are trying to predict.

```

df["selling_price"].describe()

count    1.541100e+04
mean      7.749711e+05
std       8.941284e+05
min       4.000000e+04
25%       3.850000e+05
50%       5.560000e+05
75%       8.250000e+05
max       3.950000e+07
Name: selling_price, dtype: float64

```

One important ML concept

We now have:

Features / X ————— brand model vehicle_age km_driven seller_type fuel_type transmission_type mileage engine max_power seats

and:

Target / y ————— selling_price

Our ML model will learn a relationship roughly like:

X —————→ y car information price

For example:

Hyundai + i20 + 5 years + 20,000 km + Petrol + Manual ↓ ML model ↓ ₹5–6 lakh

The actual prediction will be learned from the dataset rather than us manually creating the rule.

10. Visualize the Target Variable

A numerical summary gives us useful statistics, but a graph can show the distribution more clearly.

We will plot the distribution of `selling_price` to see:

- where most car prices are concentrated
- whether the distribution is symmetric
- whether there are very expensive outliers

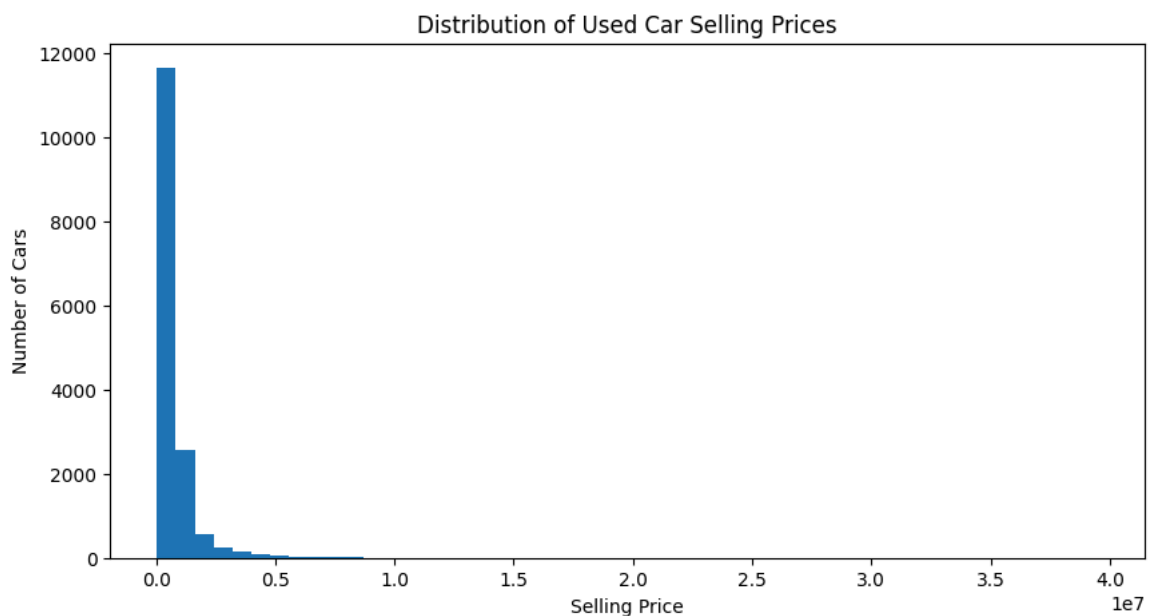
```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))

plt.hist(df["selling_price"], bins=50)

plt.xlabel("Selling Price")
plt.ylabel("Number of Cars")
plt.title("Distribution of Used Car Selling Prices")

plt.show()
```



11. Investigate High-Priced Cars

The target distribution is strongly right-skewed.

Before removing or transforming any values, we will inspect the cars with the highest selling prices.

This helps us determine whether the extreme prices are genuine observations or possible data errors.

```
df.sort_values("selling_price", ascending=False)[
    ["car_name", "brand", "model", "selling_price"]
].head(10)
```

	car_name	brand	model	selling_price
3799	Ferrari GTC4Lusso	Ferrari	GTC4Lusso	39500000
10969	Rolls-Royce Ghost	Rolls-Royce	Ghost	24200000
1172	Bentley Continental	Bentley	Continental	14500000
9722	Mercedes-Benz S-Class	Mercedes-Benz	S-Class	13000000
9364	Porsche Cayenne	Porsche	Cayenne	11100000
10989	Mercedes-Benz S-Class	Mercedes-Benz	S-Class	11000000
1888	Mercedes-Benz S-Class	Mercedes-Benz	S-Class	11000000
11000	Land Rover Rover	Land Rover	Rover	9200000
8439	BMW 7	BMW	7	8500000
3096	BMW 7	BMW	7	8500000

12. Understand Price Percentiles

The mean and maximum price can be strongly affected by a small number of expensive cars.

Percentiles help us understand the typical price range of the dataset without being dominated by extreme values.

We will inspect several percentiles of the selling price.

```
df["selling_price"].quantile(0.50)
```

```
np.float64(5560000.0)
```

```
df["selling_price"].quantile(
    [0.25, 0.50, 0.75, 0.90, 0.95, 0.99]
)
```

```
0.25    3850000.0
0.50    5560000.0
0.75    8250000.0
0.90   13750000.0
0.95   20500000.0
0.99   45450000.0
```

```
Name: selling_price, dtype: float64
```

13. Separate Features and Target

Our goal is to predict the `selling_price` of a used car.

Therefore:

- `selling_price` is our target variable (y).
- The remaining useful columns are our input features (X).

The model will learn a relationship between the car's characteristics and its selling price.

In simple terms:

Features → Model → Predicted Selling Price

```
X = df.drop(
    columns=["selling_price", "Unnamed: 0", "car_name"]
)
```

```
y = df["selling_price"]
```

```
print("Features shape:", X.shape)
print("Target shape:", y.shape)
```

```
Features shape: (15411, 11)
Target shape: (15411,)
```

14. Inspect the Model Features

Before encoding categorical variables, we will separate the features into:

1. Numerical features
2. Categorical features

This is important because different types of data require different preprocessing.

Numerical values can generally be used directly by many machine-learning models.

Categorical values such as `brand`, `fuel_type`, and `transmission_type` must be converted into a numerical representation.

```
print("Numerical columns:")
print(X.select_dtypes(include=["int64", "float64"]).columns)
```

```
print("\nCategorical columns:")
print(X.select_dtypes(include=["object"]).columns)
```

```
Numerical columns:
Index(['vehicle_age', 'km_driven', 'mileage', 'engine', 'max_power', 'seats'], dtype='object')
```

```
Categorical columns:
Index(['brand', 'model', 'seller_type', 'fuel_type', 'transmission_type'], dtype='object')
```

15. Before One-Hot Encoding

Our categorical features contain different numbers of categories:

- `brand` → 32 categories
- `model` → 120 categories
- `seller_type` → 3 categories
- `fuel_type` → 5 categories
- `transmission_type` → 2 categories

Machine-learning models generally require numerical input.

We therefore need to convert these categorical values into numerical representations.

For this project, we will use One-Hot Encoding for the categorical features.

One-Hot Encoding creates a separate binary feature for each category.

For example, if `fuel_type` contains:

Petrol, Diesel, CNG, LPG, Electric

it can become:

- `fuel_type_Petrol`
- `fuel_type_Diesel`
- `fuel_type_CNG`
- `fuel_type_LPG`
- `fuel_type_Electric`

A Petrol car would have:

Petrol = 1

and the other fuel categories would be:

Diesel = 0 CNG = 0 LPG = 0 Electric = 0

This allows the machine-learning model to work with categorical information without assuming that one category is numerically greater than another.

```
for column in X.select_dtypes(include=["object"]).columns:
    print(column, "→", X[column].nunique(), "unique values")
```

```
brand → 32 unique values
model → 120 unique values
seller_type → 3 unique values
fuel_type → 5 unique values
transmission_type → 2 unique values
```

```
categorical_columns = X.select_dtypes(include=["object"]).columns.tolist()
numerical_columns = X.select_dtypes(
    include=["int64", "float64"]
).columns.tolist()
```

```
print("Categorical columns:")
print(categorical_columns)
```

```
print("\nNumerical columns:")
print(numerical_columns)
```



```
Categorical columns:
['brand', 'model', 'seller_type', 'fuel_type', 'transmission_type']

Numerical columns:
['vehicle_age', 'km_driven', 'mileage', 'engine', 'max_power', 'seats']
```

16. Prepare the Encoding and Preprocessing

Our dataset contains both numerical and categorical features.

Numerical features can be used directly by many machine-learning models:

- `vehicle_age`
- `km_driven`
- `mileage`
- `engine`
- `max_power`
- `seats`

Categorical features contain text values:

- `brand`
- `model`
- `seller_type`
- `fuel_type`
- `transmission_type`

Machine-learning models need numerical representations, so we will use One-Hot Encoding for the categorical features.

We will use `ColumnTransformer` to apply the encoder only to categorical columns while leaving numerical columns unchanged.

Later, we will place this preprocessing inside a `Pipeline` together with our machine-learning model.

This gives us a clean sequence:

Data → Preprocessing → Model → Prediction

```
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer

preprocessor = ColumnTransformer(
    transformers=[
        (
            "categorical",
            OneHotEncoder(handle_unknown="ignore"),
            categorical_columns
        )
    ],
    remainder="passthrough"
)

print("Preprocessor created successfully.")
```

Preprocessor created successfully.

17. Understand Fit and Transform

Before using the preprocessor, we need to understand two important operations:

- `fit()` learns the required information from the data.
- `transform()` applies the learned information to data.

For One-Hot Encoding, `fit()` learns which categories exist in each categorical column.

For example, if `fuel_type` contains:

- Petrol
- Diesel
- CNG
- LPG
- Electric

the encoder learns these categories during `fit()`.

Then `transform()` uses those learned categories to convert values into numerical columns.

`fit_transform()` performs both operations together.

However, during machine-learning training we must be careful about where `fit()` is performed. We should learn preprocessing information from the training data and then use that learned preprocessing on the test data.

This prevents information from the test set from leaking into the training process.

```
example = X[["fuel_type"]].head(5)
```

```
print("Original data:")  
print(example)
```

```
Original data:  
fuel_type  
0    Petrol  
1    Petrol  
2    Petrol  
3    Petrol  
4    Diesel
```

17.1 What Does `fit()` Learn?

The encoder needs to learn which categories exist before it can convert them.

We will use only the `fuel_type` column for this small demonstration.

The `fit()` operation learns the categories from the data, but does not yet return the encoded dataset.

After fitting, we can inspect the categories learned by the encoder.

```
fuel_encoder = OneHotEncoder()
```

```
fuel_encoder.fit(example)
```

```
print("Categories learned by the encoder:")  
print(fuel_encoder.categories_)
```

```
Categories learned by the encoder:  
[array(['Diesel', 'Petrol'], dtype=object)]
```

17.2 Transform Categories into Numbers

The `fit()` operation taught the encoder which categories exist.

Now we will use `transform()` to convert the categories into numerical values.

The encoder has learned:

- Diesel
- Petrol

It will create a separate numerical column for each category.

This is the actual one-hot encoding step.

```
encoded_example = fuel_encoder.transform(example)
```

```
print("Encoded data:")  
print(encoded_example.toarray())
```

```
Encoded data:  
[[0. 1.]  
 [0. 1.]  
 [0. 1.]  
 [0. 1.]  
 [1. 0.]]
```

18. Split Data into Training and Testing

Before training the machine-learning model, we divide our dataset into two parts:

- Training data → used to teach the model.
- Testing data → used to evaluate the model on unseen data.

We will use 80% of the data for training and 20% for testing.

The target variable is `selling_price`.

The features are stored in `X` and the target is stored in `y`.

Important:

The preprocessing steps must be learned from the training data only.

This helps prevent data leakage from the test set.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("Training features:", X_train.shape)
print("Testing features:", X_test.shape)
print("Training target:", y_train.shape)
print("Testing target:", y_test.shape)
```

```
Training features: (12328, 11)
Testing features: (3083, 11)
Training target: (12328,)
Testing target: (3083,)
```

19. Fit the Preprocessor on Training Data

We have now separated the dataset into training and testing data.

The preprocessor should learn categorical information from the training data only.

Therefore:

- `fit_transform(X_train)` learns the categories from the training data and converts the training features.
- `transform(X_test)` uses the same learned categories to convert the testing features.

We do not fit the preprocessor separately on the test data.

This prevents information from the test set from leaking into the training process.

```
X_train_processed = preprocessor.fit_transform(X_train)

X_test_processed = preprocessor.transform(X_test)

print("Processed training shape:", X_train_processed.shape)
print("Processed testing shape:", X_test_processed.shape)
```

```
Processed training shape: (12328, 165)
Processed testing shape: (3083, 165)
```

20. Create the Random Forest Model

We have now completed the preprocessing stage.

The processed training data contains:

- 12,328 training records
- 165 numerical features

The processed testing data contains:

- 3,083 testing records
- 165 numerical features

Our selected machine-learning algorithm is `RandomForestRegressor`.

Random Forest is an ensemble regression algorithm. It builds many decision trees and combines their predictions to produce a final numerical prediction.

In this project, the model will learn the relationship between the car's features and its `selling_price`.

The important distinction is:

- `preprocessor` prepares the data.
- `RandomForestRegressor` learns patterns from the prepared data.
- `predict()` will later use those learned patterns to estimate selling prices.

We will start with 100 decision trees.

`random_state=42` makes the model's random behavior reproducible.

`n_jobs=-1` allows the model to use all available CPU cores where parallel processing is supported.

```
from sklearn.ensemble import RandomForestRegressor

model = RandomForestRegressor(
    n_estimators=100,
    random_state=42,
    n_jobs=-1
)

print("Random Forest model created successfully.")

Random Forest model created successfully.
```

21. Train the Random Forest Model

The Random Forest model has now been created, but it has not learned anything yet.

Training happens when we call `fit()`.

We provide:

- `X_train_processed` → the processed training features
- `y_train` → the correct selling prices for those training records

The model uses these examples to learn patterns between the car's characteristics and its selling price.

The model is trained only on the training data.

We do not use `X_test_processed` or `y_test` for training.

The workflow is:

Training data:

`X_train_processed + y_train → model.fit() → trained model`

Testing will happen later:

`X_test_processed → model.predict() → predicted prices`

This separation is important because the test data should represent unseen data when we evaluate the model.

```
model.fit(
    X_train_processed,
    y_train
)

print("Random Forest training completed.")

Random Forest training completed.
```

22. Make Predictions on the Test Data

The Random Forest model has now been trained using the training data.

We can use the trained model to predict selling prices for the test dataset.

The test features were not used during model training.

This gives us an unbiased set of predictions that we can later compare with the actual selling prices.

The workflow is:

`X_test_processed → trained Random Forest → predicted selling prices`

The resulting predictions will contain one predicted selling price for each test car.

```
y_pred = model.predict(X_test_processed)

print("Number of predictions:", len(y_pred))
print("First 10 predictions:")
print(y_pred[:10])

Number of predictions: 3083
First 10 predictions:
[ 242970.      625760.      641350.      1210700.
```

```
179869.16666667 437030. 478333.33333333 292200.
368220. 1167600. ]
```

23. Compare Actual and Predicted Prices

The model has generated predictions for all 3,083 test cars.

We now compare the predicted selling prices with the actual selling prices.

This helps us understand how closely the Random Forest model is predicting the real prices.

We will first inspect a small sample of actual and predicted values before calculating evaluation metrics.

```
comparison = pd.DataFrame({
    "Actual Price": y_test.values,
    "Predicted Price": y_pred
})

print(comparison.head(10))
```

	Actual Price	Predicted Price
0	190000	2.429700e+05
1	600000	6.257600e+05
2	665000	6.413500e+05
3	1570000	1.210700e+06
4	160000	1.798692e+05
5	675000	4.370300e+05
6	465000	4.783333e+05
7	260000	2.922000e+05
8	300000	3.682200e+05
9	850000	1.167600e+06

24. Evaluate the Random Forest Model

The model has generated predictions for all test records.

We will now evaluate the predictions using three regression metrics:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- R² Score

MAE tells us the average absolute prediction error.

RMSE gives more importance to large prediction errors.

R² measures how well the model explains the variation in selling prices.

These metrics will help us objectively evaluate the performance of the Random Forest model on unseen test data.

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

mae = mean_absolute_error(y_test, y_pred)

rmse = np.sqrt(mean_squared_error(y_test, y_pred))

r2 = r2_score(y_test, y_pred)

print("Model Evaluation")
print("-----")
print("MAE :", mae)
print("RMSE:", rmse)
print("R2 :", r2)
```

```
Model Evaluation
-----
MAE : 98825.34052979852
RMSE: 214256.11721654845
R2 : 0.9390186049329852
```

25. Visualize Actual vs Predicted Prices

We have already calculated the model evaluation metrics.

Now we will visualize the relationship between the actual selling prices and the prices predicted by the Random Forest model.

If the predictions are close to the actual values, the points should generally follow a diagonal pattern.

A perfect model would place every point exactly on the diagonal line.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 6))

plt.scatter(y_test, y_pred, alpha=0.4)

plt.xlabel("Actual Selling Price")
plt.ylabel("Predicted Selling Price")
plt.title("Actual vs Predicted Selling Prices")

# Perfect prediction reference line
min_price = min(y_test.min(), y_pred.min())
max_price = max(y_test.max(), y_pred.max())

plt.plot(
    [min_price, max_price],
    [min_price, max_price],
    linestyle="--"
)

plt.show()
```



26. Save the Trained Model and Preprocessor

Now that the Random Forest model has been trained and evaluated, we need to save it for later use.

The trained model will be used in our Django application to predict the selling price of used cars.

We also save the preprocessor because new data from the Django form must go through the same preprocessing steps that were used during model training.

The workflow in our Django application will be:

User Input → Preprocessor → Trained Random Forest Model → Predicted Car Price

We will use the `joblib` library to save both the trained model and the preprocessor as `.joblib` files.

```
import joblib

joblib.dump(model, "/kaggle/working/car_price_model.joblib")

joblib.dump(preprocessor, "/kaggle/working/preprocessor.joblib")

print("Model saved successfully.")
print("Preprocessor saved successfully.")

Model saved successfully.
Preprocessor saved successfully.
```

27. Verify the Saved Model Files

After saving the trained model and preprocessor, we should verify that both files exist successfully.

These files will later be downloaded from Kaggle and added to our Django project.

The two important files are:

- `car_price_model.joblib` — contains the trained Random Forest model.
- `preprocessor.joblib` — contains the preprocessing pipeline used to prepare input data.

It is important to keep both files because the Django application must preprocess new user input in the same way as the training data before making a prediction.

```
import os

model_path = "/kaggle/working/car_price_model.joblib"
preprocessor_path = "/kaggle/working/preprocessor.joblib"

print("Model exists:", os.path.exists(model_path))
print("Preprocessor exists:", os.path.exists(preprocessor_path))

print("\nSaved files:")

print(
    "car_price_model.joblib size:",
    os.path.getsize(model_path),
    "bytes"
)

print(
    "preprocessor.joblib size:",
    os.path.getsize(preprocessor_path),
    "bytes"
)
```

```
Model exists: True
Preprocessor exists: True

Saved files:
car_price_model.joblib size: 101653345 bytes
preprocessor.joblib size: 4750 bytes
```

28. Test the Saved Model and Preprocessor

Before using the saved model in our Django application, we should verify that the model and preprocessor can be loaded successfully.

We will load both `.joblib` files from disk and use them to make a prediction.

This simulates what will happen later in the Django application:

User Input → Preprocessor → Loaded Model → Predicted Car Price

If the loaded model can successfully make predictions, it confirms that our saved machine learning files are ready for integration.

```
import joblib

# Load the saved model
loaded_model = joblib.load("/kaggle/working/car_price_model.joblib")

# Load the saved preprocessor
loaded_preprocessor = joblib.load("/kaggle/working/preprocessor.joblib")

print("Model loaded successfully.")
print("Preprocessor loaded successfully.")

print("\nModel type:")
print(type(loaded_model))

print("\nPreprocessor type:")
print(type(loaded_preprocessor))
```

```
Model loaded successfully.
Preprocessor loaded successfully.
```

```
Model type:
<class 'sklearn.ensemble._forest.RandomForestRegressor'>
```

```
Preprocessor type:  
<class 'sklearn.compose._column_transformer.ColumnTransformer'>
```

29. Test Prediction Using the Loaded Model

We have successfully loaded the trained Random Forest model and the preprocessing pipeline from their saved `.joblib` files.

Now we will test whether the loaded files can make predictions correctly.

We will use a small sample from the test dataset and process it using the loaded preprocessor.

The processed data will then be passed to the loaded Random Forest model.

This simulates the workflow that will later be used in our Django application:

New Car Data → Loaded Preprocessor → Loaded Model → Predicted Selling Price

If the loaded model successfully produces predictions, it confirms that both saved files are working correctly and are ready to be integrated into our Django application.

```
# Take the first 5 records from the original test data  
sample_data = X_test.head(5)  
  
# Transform the data using the loaded preprocessor  
sample_processed = loaded_preprocessor.transform(sample_data)  
  
# Make predictions using the loaded model  
sample_predictions = loaded_model.predict(sample_processed)  
  
# Display the predictions  
for i, prediction in enumerate(sample_predictions, start=1):  
    print(f"Car {i} predicted price: ₹ {prediction:,.2f}")  
  
Car 1 predicted price: ₹ 242,970.00  
Car 2 predicted price: ₹ 625,760.00  
Car 3 predicted price: ₹ 641,350.00  
Car 4 predicted price: ₹ 1,210,700.00  
Car 5 predicted price: ₹ 179,869.17
```

30. Create a Reusable Prediction Function

To prepare our machine learning model for Django integration, we will create a reusable prediction function.

The function will:

1. Accept new car data
2. Apply the saved preprocessor
3. Use the trained model
4. Return the predicted selling price

This structure will later make it easier to connect our machine learning model with a Django form and web interface.

```
import pandas as pd  
  
def predict_car_price(car_data):  
    # Convert input data into a DataFrame  
    input_df = pd.DataFrame([car_data])  
  
    # Apply preprocessing  
    processed_data = loaded_preprocessor.transform(input_df)  
  
    # Make prediction  
    prediction = loaded_model.predict(processed_data)  
  
    # Return predicted price  
    return prediction[0]
```

31. Test the Reusable Prediction Function

We will now test the reusable prediction function using one car record from the test dataset.

This confirms that the function can:

1. Accept car information as input
2. Convert the information into a DataFrame

3. Apply the saved preprocessor
4. Generate a prediction using the trained model
5. Return the predicted selling price

This is very similar to how our Django application will later send user-entered car information to the machine learning model.

```
# Take one car record from the test dataset
sample_car = X_test.iloc[0].to_dict()

# Predict the selling price
predicted_price = predict_car_price(sample_car)

print("Predicted selling price: ₹", round(predicted_price, 2))
print("Actual selling price: ₹", round(y_test.iloc[0], 2))
```

```
Predicted selling price: ₹ 242970.0
Actual selling price: ₹ 190000
```

add new data set

32. Load the New Used-Car Dataset
33. Understand the New Dataset
34. Compare Old vs New Dataset
35. Check Missing Values
36. Check Duplicates
37. Validate Year and Age
38. Clean kmDriven
39. Analyze AskPrice
40. Investigate Suspicious Records
41. Prepare Features and Target ...

32. Load the New Used-Car Dataset

In the previous sections, we worked with the CarDekho dataset and built our baseline used-car price prediction model.

Now we will investigate a second Indian used-car dataset.

The purpose of this section is **not to immediately replace the existing model**.

First, we need to understand the new dataset:

- What columns does it contain?
- How many rows are available?
- What type of data does each column contain?
- Is the target variable suitable for regression?
- Are there missing or duplicate records?
- Does the dataset contain useful information that can improve our project?

This step is important because a machine learning model is only as reliable as the data used to train it.

```
import pandas as pd

new_dataset_path = "/kaggle/input/datasets/mohitkumar282/used-car-dataset/used_cars_dataset_v2.csv"

new_df = pd.read_csv(new_dataset_path)

print("Dataset loaded successfully.")
print("Shape:", new_df.shape)

display(new_df.head())
```

```
Dataset loaded successfully.
Shape: (14993, 11)
```

	Brand	model	Year	Age	kmDriven	Transmission	Owner	FuelType	PostedDate	AdditionInfo	AskPrice
0	Honda	City	2001	23	98,000 km	Manual	second	Petrol	Nov-24	Honda City v teck in mint condition, valid gen...	₹ 1,95,000
1	Toyota	Innova	2009	15	190000.0 km	Manual	second	Diesel	Jul-24	Toyota Innova 2.5 G (Diesel) 7 Seater, 2009, D...	₹ 3,75,000
2	Volkswagen	Vento	2010	14	77,246 km	Manual	first	Diesel	Nov-24	Volkswagen Vento 2010-2012 Diesel	₹ 1,94,000

33. Understand the New Dataset

Before cleaning or training a machine learning model, we need to understand the structure of the dataset.

We will inspect:

- The number of rows and columns
- The names of all columns
- The data types
- A summary of the dataset structure

Understanding the schema helps us identify which columns can be used as features and which column should become our target variable.

For this dataset, `AskPrice` represents the asking price of the used car, so it is a possible target variable for our regression problem.

```
print("Number of rows:", new_df.shape[0])
print("Number of columns:", new_df.shape[1])
```

```
print("\nColumn names:")
print(new_df.columns.tolist())
```

```
print("\nDataset information:")
new_df.info()
```

```
Number of rows: 14993
Number of columns: 11
```

```
Column names:
['Brand', 'model', 'Year', 'Age', 'kmDriven', 'Transmission', 'Owner', 'FuelType', 'PostedDate', 'AdditionInfo',
```

```
Dataset information:
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 14993 entries, 0 to 14992
```

```
Data columns (total 11 columns):
```

#	Column	Non-Null Count	Dtype
0	Brand	14993 non-null	object
1	model	14993 non-null	object
2	Year	14993 non-null	int64
3	Age	14993 non-null	int64
4	kmDriven	14905 non-null	object
5	Transmission	14993 non-null	object
6	Owner	14993 non-null	object
7	FuelType	14993 non-null	object
8	PostedDate	14993 non-null	object
9	AdditionInfo	14993 non-null	object
10	AskPrice	14993 non-null	object

```
dtypes: int64(2), object(9)
```

```
memory usage: 1.3+ MB
```

34. Compare the Old and New Dataset

We already investigated the CarDekho dataset in the previous sections.

Before deciding whether the new dataset should be used for model training, we should compare the two datasets.

The comparison will help us understand:

- Dataset size
- Number of features
- Available vehicle information
- Target variable
- Data format

This is important because a larger dataset is not automatically a better dataset. Data quality and useful features are also important.

```
print("OLD DATASET - CARDEKHO")
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
print("Target column: selling_price")
```

```
print("\nNEW DATASET - USED CAR DATASET V2")
print("Rows:", new_df.shape[0])
print("Columns:", new_df.shape[1])
print("Target column: AskPrice")
```

```
print("\nOLD DATASET COLUMNS:")
print(df.columns.tolist())
```

```
print("\nNEW DATASET COLUMNS:")
print(new_df.columns.tolist())
```

```
OLD DATASET - CARDEKHO
Rows: 15411
Columns: 14
Target column: selling_price
```

```
NEW DATASET - USED CAR DATASET V2
Rows: 14993
Columns: 11
Target column: AskPrice
```

```
OLD DATASET COLUMNS:
['Unnamed: 0', 'car_name', 'brand', 'model', 'vehicle_age', 'km_driven', 'seller_type', 'fuel_type', 'transmission', 'owner_type', 'newspaper_dealership', 'big_dealership', 'dealer_type']

NEW DATASET COLUMNS:
['Brand', 'model', 'Year', 'Age', 'kmDriven', 'Transmission', 'Owner', 'FuelType', 'PostedDate', 'AdditionInfo', 'AskPrice']
```

35. Check Missing Values

Missing values occur when information is not available for a particular record.

Missing data can cause problems during machine learning because many algorithms require numerical or categorical values to be present.

We will check the new dataset for missing values before deciding how to clean the data.

It is important to distinguish between:

- **Missing values:** information is actually absent.
- **Invalid values:** information exists but is incorrect or unusable.

A dataset can have zero missing values and still contain invalid records.

```
print("Missing values in each column:")
print(new_df.isnull().sum())

print("\nTotal missing values:")
print(new_df.isnull().sum().sum())
```

```
Missing values in each column:
Brand      0
model      0
Year       0
Age        0
kmDriven   88
Transmission 0
Owner      0
FuelType   0
PostedDate 0
AdditionInfo 0
AskPrice   0
dtype: int64
```

```
Total missing values:
88
```

36. Check Duplicate Records

Duplicate records occur when the same car listing appears more than once in the dataset.

Duplicates can affect machine learning because repeated records may give some examples more influence during training.

We will check for exact duplicate rows before performing any cleaning.

At this stage, we are only measuring duplicates. We are not removing them yet.

```
duplicate_count = new_df.duplicated().sum()

print("Number of duplicate rows:", duplicate_count)

print("\nPercentage of duplicate rows:")
print((duplicate_count / len(new_df)) * 100)
```

```
Number of duplicate rows: 997
```

```
Percentage of duplicate rows:  
6.649769892616554
```

37. Validate Year and Age

The dataset contains two related numerical features:

- **Year** — the manufacturing year of the car
- **Age** — the age of the car

These two values should have a logical relationship.

For example, if a car was manufactured in 2020 and the dataset uses 2024 as the reference year, its age should be:

```
2024 - 2020 = 4 years
```

We will check whether the **Age** column is consistent with the manufacturing **Year**.

This is an example of **data validation**.

A value can be non-missing and have the correct data type while still being logically incorrect.

```
reference_year = 2024  
  
new_df["CalculatedAge"] = reference_year - new_df["Year"]  
  
age_mismatch = new_df[new_df["Age"] != new_df["CalculatedAge"]]  
  
print("Reference year:", reference_year)  
print("Total records:", len(new_df))  
print("Age mismatches:", len(age_mismatch))  
  
print("\nPercentage of age mismatches:")  
print((len(age_mismatch) / len(new_df)) * 100)  
  
print("\nSample mismatched records:")  
display(age_mismatch[["Brand", "model", "Year", "Age", "CalculatedAge"]].head(10))
```

```
Reference year: 2024  
Total records: 14993  
Age mismatches: 0
```

```
Percentage of age mismatches:  
0.0
```

```
Sample mismatched records:  
  Brand model Year Age CalculatedAge
```

38. Inspect Manufacturing Year

The manufacturing year is an important feature for used-car price prediction.

Generally, newer cars tend to have higher prices than older cars, although brand, model, mileage, fuel type, transmission, and other factors also affect price.

Before using **Year** for machine learning, we should inspect its range and distribution.

This helps us identify potentially invalid or suspicious manufacturing years.

```
print("Minimum manufacturing year:", new_df["Year"].min())  
print("Maximum manufacturing year:", new_df["Year"].max())  
  
print("\nManufacturing year distribution:")  
print(new_df["Year"].value_counts().sort_index())
```

```
Minimum manufacturing year: 1900  
Maximum manufacturing year: 2024
```

```
Manufacturing year distribution:  
Year  
1900      1  
1920      1  
1931      1  
1978      1  
1986      1  
1990      1  
1994      3  
1996      2
```

```

1997      2
1998      3
1999      5
2000      6
2001     10
2002      8
2003     18
2004     27
2005     56
2006     60
2007    120
2008    161
2009    276
2010    532
2011    681
2012    726
2013    798
2014    991
2015   1293
2016   1421
2017   1598
2018   1549
2019   1298
2020    855
2021    877
2022    827
2023    523
2024    261
Name: count, dtype: int64

```

39. Investigate Suspicious Manufacturing Years

The previous section showed that the dataset contains some unusually old manufacturing years.

An old year is not automatically an error because genuine vintage and older used cars can exist.

Therefore, instead of deleting these records based only on the year, we will inspect the associated brand, model, fuel type, transmission, age, and asking price.

This allows us to identify records that are genuinely suspicious based on their overall information.

```

suspicious_years = new_df[new_df["Year"] < 2000]

print("Records with manufacturing year before 2000:")
print("Count:", len(suspicious_years))

display(
    suspicious_years[
        [
            "Brand",
            "model",
            "Year",
            "Age",
            "kmDriven",
            "Transmission",
            "Owner",
            "FuelType",
            "AskPrice"
        ]
    ].sort_values("Year")
)

```

Records with manufacturing year before 2000:
Count: 21

	Brand	model	Year	Age	kmDriven	Transmission	Owner	FuelType	AskPrice
11392	Renault	Triber	1900	124	40,000 km	Automatic	second	hybrid	₹ 4,40,000
12545	Maruti Suzuki	Swift	1920	104	60,000 km	Manual	first	Petrol	₹ 5,19,000
11260	Maruti Suzuki	Wagon-R	1931	93	88,000 km	Manual	first	hybrid	₹ 3,49,000
11458	Mahindra	Jeep	1978	46	100,000 km	Manual	second	Diesel	₹ 2,00,000
7404	Maruti Suzuki	800	1986	38	68,000 km	Manual	second	Petrol	₹ 1,20,000
1768	Mahindra	Jeep	1990	34	9,000 km	Manual	second	Diesel	₹ 2,50,000
6360	Maruti Suzuki	800	1994	30	70,120 km	Automatic	second	Petrol	₹ 35,000
508	Mahindra	Jeep	1994	30	0 km	Manual	second	Diesel	₹ 2,75,000
7622	Mahindra	Jeep	1994	30	0 km	Manual	second	Diesel	₹ 2,95,000
6828	Tata	Sumo	1996	28	110,000 km	Manual	second	Diesel	₹ 1,50,000
2802	Mahindra	Jeep	1996	28	50,000 km	Manual	second	Diesel	₹ 2,99,000
1390	Maruti Suzuki	800	1997	27	28,000 km	Manual	first	Petrol	₹ 60,000
8068	Mahindra	Jeep	1997	27	225,897 km	Manual	second	Diesel	₹ 1,90,000
7242	Maruti Suzuki	Zen-Estilo	1998	26	NaN	Manual	second	Petrol	₹ 27,000
6232	Maruti Suzuki	800	1998	26	98,000 km	Manual	second	Petrol	₹ 41,000
204	Maruti Suzuki	Omni	1998	26	80,000 km	Manual	second	Petrol	₹ 75,000
8793	Mahindra	Jeep	1999	25	78,000 km	Manual	second	Diesel	₹ 80,000
7763	Toyota	Land Cruiser	1999	25	30,000 km	Automatic	first	Petrol	₹ 21,00,000
2838	Mitsubishi	Lancer	1999	25	225,000 km	Manual	second	Diesel	₹ 90,000
3493	Hyundai	Alcazar	1999	25	16,000 km	Manual	first	Petrol	₹ 15,95,000
14162	Hyundai	Santro	1999	25	150,000 km	Manual	second	hybrid	₹ 74,999

40. Check Suspicious Model and Year Combinations

A manufacturing year can only be interpreted correctly when it is consistent with the vehicle model.

Some records in the dataset have unusually old manufacturing years. Instead of removing all old vehicles, we will inspect whether the model itself appears to be a modern vehicle.

This helps us identify possible data-entry or scraping errors.

We will focus on records with a manufacturing year before 2000 and display their brand, model, year, age, fuel type, and asking price.

```
print("Potentially suspicious model/year combinations:")

display(
    suspicious_years[
        [
            "Brand",
            "model",
            "Year",
            "Age",
            "FuelType",
            "AskPrice"
        ]
    ].sort_values(["Brand", "model", "Year"])
)
```

Potentially suspicious model/year combinations:

	Brand	model	Year	Age	FuelType	AskPrice
3493	Hyundai	Alcazar	1999	25	Petrol	₹ 15,95,000
14162	Hyundai	Santro	1999	25	hybrid	₹ 74,999
11458	Mahindra	Jeep	1978	46	Diesel	₹ 2,00,000
1768	Mahindra	Jeep	1990	34	Diesel	₹ 2,50,000
508	Mahindra	Jeep	1994	30	Diesel	₹ 2,75,000
7622	Mahindra	Jeep	1994	30	Diesel	₹ 2,95,000
2802	Mahindra	Jeep	1996	28	Diesel	₹ 2,99,000
8068	Mahindra	Jeep	1997	27	Diesel	₹ 1,90,000
8793	Mahindra	Jeep	1999	25	Diesel	₹ 80,000
7404	Maruti Suzuki	800	1986	38	Petrol	₹ 1,20,000
6360	Maruti Suzuki	800	1994	30	Petrol	₹ 35,000
1390	Maruti Suzuki	800	1997	27	Petrol	₹ 60,000
6232	Maruti Suzuki	800	1998	26	Petrol	₹ 41,000
204	Maruti Suzuki	Omni	1998	26	Petrol	₹ 75,000
12545	Maruti Suzuki	Swift	1920	104	Petrol	₹ 5,19,000
11260	Maruti Suzuki	Wagon-R	1931	93	hybrid	₹ 3,49,000
7242	Maruti Suzuki	Zen-Estilo	1998	26	Petrol	₹ 27,000
2838	Mitsubishi	Lancer	1999	25	Diesel	₹ 90,000
11392	Renault	Triber	1900	124	hybrid	₹ 4,40,000
6828	Tata	Sumo	1996	28	Diesel	₹ 1,50,000
7763	Toyota	Land Cruiser	1999	25	Petrol	₹ 21,00,000

41. Identify Clearly Invalid Records

After investigating the unusually old vehicles, we found that some records are clearly inconsistent with the vehicle model and manufacturing year.

We should not remove all old vehicles because some older cars are legitimate.

Instead, we will identify only records that are clearly impossible or strongly inconsistent.

For this dataset, the following records will be treated as invalid:

- Renault Triber with manufacturing year 1900
- Maruti Suzuki Swift with manufacturing year 1920
- Maruti Suzuki Wagon-R with manufacturing year 1931
- Hyundai Alcazar with manufacturing year 1999
- Hyundai Santro with manufacturing year 1999 and hybrid fuel type

We will first count these records and inspect them before creating the cleaned dataset.

The original `new_df` DataFrame will remain unchanged.

```
invalid_records = (
    ((new_df["Brand"] == "Renault") & (new_df["model"] == "Triber") & (new_df["Year"] == 1900))
    | ((new_df["Brand"] == "Maruti Suzuki") & (new_df["model"] == "Swift") & (new_df["Year"] == 1920))
    | ((new_df["Brand"] == "Maruti Suzuki") & (new_df["model"] == "Wagon-R") & (new_df["Year"] == 1931))
    | ((new_df["Brand"] == "Hyundai") & (new_df["model"] == "Alcazar") & (new_df["Year"] == 1999))
    | ((new_df["Brand"] == "Hyundai") & (new_df["model"] == "Santro") & (new_df["Year"] == 1999) & (new_df["Fuel
    ])

print("Clearly invalid records:", invalid_records.sum())

print("\nRecords that would be removed:")
display(
    new_df.loc[
        invalid_records,
        ["Brand", "model", "Year", "Age", "FuelType", "AskPrice"]
    ]
)
```

Clearly invalid records: 5

Records that would be removed:

	Brand	model	Year	Age	FuelType	AskPrice
3493	Hyundai	Alcazar	1999	25	Petrol	₹ 15,95,000
11260	Maruti Suzuki	Wagon-R	1931	93	hybrid	₹ 3,49,000
11392	Renault	Triber	1900	124	hybrid	₹ 4,40,000
12545	Maruti Suzuki	Swift	1920	104	Petrol	₹ 5,19,000
14162	Hyundai	Santro	1999	25	hybrid	₹ 74,999

42. Create the Cleaned Dataset

We have completed the initial data-quality investigation.

We identified five records that are clearly inconsistent with the vehicle model and manufacturing year.

Instead of modifying the original dataset, we will create a separate cleaned DataFrame.

Keeping the original dataset unchanged is useful because it allows us to:

- Compare the raw and cleaned datasets
- Reproduce our cleaning process
- Avoid accidentally losing original information
- Clearly document which records were removed

The cleaned dataset will be used for the next stages of data preparation and machine learning.

```
clean_df = new_df.loc[~invalid_records].copy()

print("Original number of records:", len(new_df))
print("Removed invalid records:", invalid_records.sum())
print("Cleaned number of records:", len(clean_df))

print("\nRemaining records:")
print(clean_df.shape)
```

```
Original number of records: 14993
Removed invalid records: 5
Cleaned number of records: 14988
```

```
Remaining records:
(14988, 12)
```

43. Remove the Temporary Audit Column

In Section 37, we created `CalculatedAge` only to validate the relationship between `Year` and `Age`.

The validation showed zero mismatches, so this temporary column is no longer needed.

We will remove it from the cleaned dataset.

The original `new_df` DataFrame is still preserved.

```
clean_df = clean_df.drop(columns=["CalculatedAge"])

print("Cleaned dataset shape:", clean_df.shape)

print("\nCleaned dataset columns:")
print(clean_df.columns.tolist())
```

```
Cleaned dataset shape: (14988, 11)
```

```
Cleaned dataset columns:
['Brand', 'model', 'Year', 'Age', 'kmDriven', 'Transmission', 'Owner', 'FuelType', 'PostedDate', 'AdditionInfo',
```

44. Inspect `kmDriven` Before Cleaning

The `kmDriven` column represents the distance travelled by the vehicle.

Although this is a numerical feature, the raw dataset stores it as text.

For example:


```
"98,000 km"
"190,000 km"
"77,246 km"
```

Machine learning algorithms cannot directly use these formatted text values as numerical measurements.

Before converting the column, we will inspect its values and determine how they are represented.

We will also check how many values are missing.

```
print("kmDriven data type:", clean_df["kmDriven"].dtype)

print("\nSample kmDriven values:")
display(clean_df["kmDriven"].head(10))

print("\nMissing kmDriven values:")
print(clean_df["kmDriven"].isnull().sum())

print("\nNumber of unique kmDriven values:")
print(clean_df["kmDriven"].nunique())
```

```
kmDriven data type: object

Sample kmDriven values:
0      98,000 km
1    190000.0 km
2     77,246 km
3     83,500 km
4     45,000 km
5     83,000 km
6    168,000 km
7     25,000 km
8     33,759 km
9     69,713 km
Name: kmDriven, dtype: object

Missing kmDriven values:
88

Number of unique kmDriven values:
2718
```

✓ 45. Convert `kmDriven` to Numeric

The `kmDriven` column is currently stored as text.

The values contain formatting characters such as commas and the `km` unit. These characters must be removed before machine learning can use the feature as a numerical variable.

For example:

```
"98,000 km" → 98000
```

We will:

1. Remove the comma separator.
2. Remove the `km` unit.
3. Convert the cleaned values to numeric.
4. Preserve missing values as `NaN`.

We will create a new numeric version of the column so that the original cleaned dataset remains easy to understand and audit.

```
# Create a numeric version of kmDriven
clean_df["kmDriven_numeric"] = (
    clean_df["kmDriven"]
    .str.replace(",", "", regex=False)
    .str.replace(" km", "", regex=False)
)

clean_df["kmDriven_numeric"] = pd.to_numeric(
    clean_df["kmDriven_numeric"],
    errors="coerce"
)

print("Original kmDriven data type:", clean_df["kmDriven"].dtype)
print("New kmDriven_numeric data type:", clean_df["kmDriven_numeric"].dtype)
```

```
print("\nMissing values after conversion:")
print(clean_df["kmDriven_numeric"].isnull().sum())

print("\nSample conversion:")
display(
    clean_df[["kmDriven", "kmDriven_numeric"]].head(10)
)
```

Original kmDriven data type: object
New kmDriven_numeric data type: float64

Missing values after conversion:
88

Sample conversion:

	kmDriven	kmDriven_numeric
0	98,000 km	98000.0
1	190000.0 km	190000.0
2	77,246 km	77246.0
3	83,500 km	83500.0
4	45,000 km	45000.0
5	83,000 km	83000.0
6	168,000 km	168000.0
7	25,000 km	25000.0
8	33,759 km	33759.0
9	69,713 km	69713.0

46. Verify the `kmDriven` Conversion

The conversion created a numerical version of the mileage feature.

Before replacing the original column, we should verify that:

- The converted column is numeric.
- Valid mileage values were converted correctly.
- The number of missing values has not unexpectedly increased.
- No negative mileage values were created.

This is a validation step before continuing with data cleaning.

```
print("Data type:", clean_df["kmDriven_numeric"].dtype)

print("\nMissing values:", clean_df["kmDriven_numeric"].isnull().sum())

print("\nMinimum kmDriven:", clean_df["kmDriven_numeric"].min())
print("Maximum kmDriven:", clean_df["kmDriven_numeric"].max())

print("\nNegative kmDriven values:")
print((clean_df["kmDriven_numeric"] < 0).sum())

print("\nSummary statistics:")
print(clean_df["kmDriven_numeric"].describe())
```

Data type: float64

Missing values: 88

Minimum kmDriven: 0.0
Maximum kmDriven: 980002.0

Negative kmDriven values:
0

Summary statistics:

count	14900.000000
mean	71078.441007
std	57461.714589
min	0.000000
25%	44000.000000
50%	65200.000000
75%	86000.000000
max	980002.000000

Name: kmDriven_numeric, dtype: float64

47. Inspect Missing `kmDriven` Records

The numerical conversion preserved 88 missing `kmDriven` values.

Before deciding how to handle these records, we should inspect the affected cars.

Removing records without investigation could unnecessarily reduce the dataset. On the other hand, keeping missing mileage values may require a suitable missing-value strategy before machine learning.

We will therefore inspect the records first and then decide whether removal or imputation is appropriate.

```
missing_km_records = clean_df[clean_df["kmDriven_numeric"].isnull()]

print("Records with missing kmDriven:", len(missing_km_records))

display(
    missing_km_records[
        [
            "Brand",
            "model",
            "Year",
            "Age",
            "Transmission",
            "Owner",
            "FuelType",
            "AskPrice"
        ]
    ].head(20)
)
```

Records with missing kmDriven: 88

	Brand	model	Year	Age	Transmission	Owner	FuelType	AskPrice
231	Maruti Suzuki	Alto-800	2019	5	Manual	first	Petrol	₹ 3,00,000
353	Toyota	Innova	2016	8	Manual	second	Diesel	₹ 8,60,000
444	Honda	City ZX	2024	0	Automatic	first	Petrol	₹ 15,50,000
476	Hyundai	Santro Xing	2010	14	Automatic	second	Petrol	₹ 1,20,000
608	Toyota	Etios	2012	12	Manual	second	Diesel	₹ 3,30,000
793	Skoda	Kushaq	2010	14	Automatic	first	Petrol	₹ 4,00,000
976	Mahindra	Bolero Neo Plus	2015	9	Automatic	first	Diesel	₹ 4,50,000
1164	Mahindra	Bolero Neo	2023	1	Manual	first	Diesel	₹ 10,00,000
1199	Tata	Indica	2014	10	Manual	first	Diesel	₹ 2,50,000
1413	Hyundai	Santro Xing	2007	17	Manual	second	Petrol	₹ 1,48,000
1978	Tata	Tiago Nrg	2021	3	Automatic	first	Petrol	₹ 4,30,000
2680	Maruti Suzuki	Zen-Estilo	2008	16	Automatic	second	Petrol	₹ 1,80,000
2847	Maruti Suzuki	S-Presso	2021	3	Automatic	first	Petrol	₹ 4,25,000
3009	Maruti Suzuki	Baleno	2018	6	Manual	first	Petrol	₹ 4,85,000
3361	Ambassador	Ambassador	2008	16	Manual	first	Diesel	₹ 1,50,000
3418	Honda	City	2001	23	Manual	second	Petrol	₹ 55,000
3749	Mahindra	Bolero Power Plus	2015	9	Automatic	second	Diesel	₹ 4,50,000
3869	Maruti Suzuki	Esteem	2007	17	Manual	second	Hybrid/CNG	₹ 1,00,000
3922	Maruti Suzuki	Omni	2008	16	Manual	second	Hybrid/CNG	₹ 51,786
4192	ICML	Rhino Rx	2010	14	Manual	second	Diesel	₹ 1,95,000

48. Analyze Missing `kmDriven` by Manufacturing Year

The previous section showed that 88 records have missing mileage.

Before deciding how to handle these records, we can examine whether the missing values are concentrated in particular manufacturing years.

This helps us understand whether the missing mileage is random or whether certain groups of vehicles are affected more often.

We will compare the number of missing mileage records across manufacturing years.

```
missing_km_by_year = (
    missing_km_records["Year"]
    .value_counts()
    .sort_index()
)

print("Missing kmDriven records by manufacturing year:")
print(missing_km_by_year)
```

```
Missing kmDriven records by manufacturing year:
Year
1998    1
2001    1
2002    1
2004    1
2005    2
2006    1
2007    5
2008    3
2009    3
2010    9
2011    4
2012    4
2013    7
2014    4
2015    7
2016    6
2017    3
2018    6
2019    9
2020    2
2021    2
2022    2
2023    4
2024    1
Name: count, dtype: int64
```

49. Measure the Missing `kmDriven` Rate

We found 88 records with missing mileage.

The number of missing records alone does not tell us how significant the problem is.

We will calculate the percentage of records with missing `kmDriven` values.

This helps us determine whether the missing-data problem is small enough to handle without removing a large portion of the dataset.

```
total_records = len(clean_df)

missing_km_count = clean_df["kmDriven_numeric"].isnull().sum()

missing_km_percentage = (
    missing_km_count / total_records
) * 100

print("Total records:", total_records)
print("Missing kmDriven records:", missing_km_count)
print("Missing kmDriven percentage:", round(missing_km_percentage, 2), "%")
```

```
Total records: 14988
Missing kmDriven records: 88
Missing kmDriven percentage: 0.59 %
```

50. Decide How to Handle Missing `kmDriven`

We found 88 records with missing `kmDriven`, representing only 0.59% of the cleaned dataset.

Because the missing portion is small, removing these records is possible, but it would unnecessarily discard valid vehicle information.

Before choosing a strategy, we will examine whether mileage can be estimated using similar vehicles.

A vehicle's mileage is generally related to factors such as its manufacturing year, brand, and model.

We will therefore compare missing-mileage records with cars having similar manufacturing years and models.

For the machine-learning pipeline, the final handling of missing values will be performed without using information from the test set.

```
# Select records where kmDriven is missing
missing_km_records = clean_df[
    clean_df["kmDriven_numeric"].isnull()
].copy()
```

```

print("Missing kmDriven records:", len(missing_km_records))

# Check whether these records have usable information
print("\nMissing records with known Brand:")
print(missing_km_records["Brand"].notnull().sum())

print("\nMissing records with known model:")
print(missing_km_records["model"].notnull().sum())

print("\nMissing records with known Year:")
print(missing_km_records["Year"].notnull().sum())

print("\nSample missing-mileage records:")
display(
    missing_km_records[
        [
            "Brand",
            "model",
            "Year",
            "Age",
            "Transmission",
            "Owner",
            "FuelType",
            "AskPrice"
        ]
    ].head(10)
)

```

Missing kmDriven records: 88

Missing records with known Brand:
88

Missing records with known model:
88

Missing records with known Year:
88

Sample missing-mileage records:

	Brand	model	Year	Age	Transmission	Owner	FuelType	AskPrice
231	Maruti Suzuki	Alto-800	2019	5	Manual	first	Petrol	₹ 3,00,000
353	Toyota	Innova	2016	8	Manual	second	Diesel	₹ 8,60,000
444	Honda	City ZX	2024	0	Automatic	first	Petrol	₹ 15,50,000
476	Hyundai	Santro Xing	2010	14	Automatic	second	Petrol	₹ 1,20,000
608	Toyota	Etios	2012	12	Manual	second	Diesel	₹ 3,30,000
793	Skoda	Kushaq	2010	14	Automatic	first	Petrol	₹ 4,00,000
976	Mahindra	Bolero Neo Plus	2015	9	Automatic	first	Diesel	₹ 4,50,000
1164	Mahindra	Bolero Neo	2023	1	Manual	first	Diesel	₹ 10,00,000
1199	Tata	Indica	2014	10	Manual	first	Diesel	₹ 2,50,000
1413	Hyundai	Santro Xing	2007	17	Manual	second	Petrol	₹ 1,48,000

51. Check Mileage Availability for Similar Cars

The missing `kmDriven` records contain valid Brand, model, and Year information.

Before imputing the missing mileage values, we will check whether similar vehicles in the dataset have known mileage.

We will first compare cars using the same Brand and model.

If enough matching vehicles have known mileage, their mileage can provide a more meaningful estimate than using the overall dataset median.

This is an exploratory step. The final imputation strategy will be applied only after the available matching records have been evaluated.

```

# Count cars with known mileage for each Brand and model
known_km_records = clean_df[
    clean_df["kmDriven_numeric"].notnull()
].copy()

model_km_counts = (
    known_km_records
    .groupby(["Brand", "model"])["kmDriven_numeric"]

```

```
.count()
.reset_index(name="known_km_count")
)

# Attach the number of known-mileage cars to each missing-mileage record
missing_km_analysis = missing_km_records.merge(
    model_km_counts,
    on=["Brand", "model"],
    how="left"
)

# Missing combinations with no known mileage records
missing_km_analysis["known_km_count"] = (
    missing_km_analysis["known_km_count"]
    .fillna(0)
    .astype(int)
)

print("Missing-mileage records:", len(missing_km_analysis))

print("\nRecords with at least one known-mileage similar model:")
print(
    (missing_km_analysis["known_km_count"] > 0).sum()
)

print("\nRecords with no known-mileage similar model:")
print(
    (missing_km_analysis["known_km_count"] == 0).sum()
)

print("\nSample analysis:")
display(
    missing_km_analysis[
        [
            "Brand",
            "model",
            "Year",
            "Age",
            "known_km_count"
        ]
    ].head(20)
)
```

Missing-mileage records: 88

Records with at least one known-mileage similar model:
86

Records with no known-mileage similar model:
2

Sample analysis:

	Brand	model	Year	Age	known_km_count
0	Maruti Suzuki	Alto-800	2019	5	195
1	Toyota	Innova	2016	8	154
2	Honda	City ZX	2024	0	56
3	Hyundai	Santro Xing	2010	14	105
4	Toyota	Etios	2012	12	64
5	Skoda	Kushaq	2010	14	22
6	Mahindra	Bolero Neo Plus	2015	9	2
7	Mahindra	Bolero Neo	2023	1	9
8	Tata	Indica	2014	10	6
9	Hyundai	Santro Xing	2007	17	105
10	Tata	Tiago Nrg	2021	3	2
11	Maruti Suzuki	Zen-Estilo	2008	16	34
12	Maruti Suzuki	S-Presso	2021	3	47
13	Maruti Suzuki	Baleno	2018	6	280
14	Ambassador	Ambassador	2008	16	4
15	Honda	City	2001	23	550
16	Mahindra	Bolero Power Plus	2015	9	8
17	Maruti Suzuki	Esteem	2007	17	3
18	Maruti Suzuki	Omni	2008	16	81
19	ICML	Rhino Rx	2010	14	0

52. Calculate Typical Mileage for Each Model

Most missing-mileage records have other cars from the same Brand and model with known mileage.

We will calculate the median mileage for each Brand and model.

The median is preferred because it is less affected by extreme mileage values than the mean.

These model-level medians will later be used as the first option for imputing missing mileage values.

```
# Calculate the median known mileage for each Brand and model
model_km_median = (
    known_km_records
    .groupby(["Brand", "model"])["kmDriven_numeric"]
    .median()
    .reset_index(name="model_median_km")
)

print("Number of Brand + model groups with known mileage:")
print(len(model_km_median))

print("\nSample model-level mileage medians:")

display(
    model_km_median.head(20)
)
```

Number of Brand + model groups with known mileage:
435

Sample model-level mileage medians:

	Brand	model	model_median_km
0	Ambassador	Ambassador	27009.0
1	Ashok	Leyland Stile	0.0
2	Aston Martin	Vanquish	25700.0
3	Audi	2.9 Sportback	44000.0
4	Audi	A3	42000.0
5	Audi	A3 Cabriolet	37909.0
6	Audi	A4	73000.0
7	Audi	A5	90000.0
8	Audi	A6	68988.5
9	Audi	A8	77665.0
10	Audi	A8 L	55000.0
11	Audi	Q2	26207.0
12	Audi	Q3	77761.0
13	Audi	Q3 Sportback	1100.0
14	Audi	Q5	91500.0
15	Audi	Q7	45000.0
16	Audi	RS7	125000.0
17	BMW	1 Series	69000.0
18	BMW	2 Series	14000.0
19	BMW	3 Series	58000.0

53. Inspect Extreme Model-Level Mileage Medians

The model-level median calculation produced mileage estimates for 435 Brand and model groups.

Before using these medians for missing-value imputation, we should inspect unusually low and unusually high values.

Very low mileage may represent new or demo vehicles, while very high mileage may indicate unusual usage or possible data-quality problems.

We will inspect these groups before deciding whether the model-level median is suitable for imputation.

```
# Inspect model groups with unusually low median mileage
low_median_models = model_km_median[
    model_km_median["model_median_km"] < 5000
].sort_values("model_median_km")

print("Model groups with median mileage below 5,000 km:")
display(low_median_models)
```

Model groups with median mileage below 5,000 km:

	Brand	model	model_median_km
1	Ashok	Leyland Stile	0.0
145	Kia	CERATO	0.0
197	Mahindra	Thar Roxx	0.0
339	Rolls-Royce	Phantom Drop HeadCoupe	2.0
175	Mahindra	Imperio	10.0
174	Mahindra	Five-door Thar	45.0
372	Tata	Punch EV	225.0
13	Audi	Q3 Sportback	1100.0
291	Mercedes-Benz	SL-Class	1400.0
233	Maruti Suzuki	Jimny	1800.0
201	Mahindra	XUV 3XO	2500.0


```
# Inspect model groups with unusually high median mileage
high_median_models = model_km_median[
    model_km_median["model_median_km"] > 200000
].sort_values("model_median_km", ascending=False)

print("Model groups with median mileage above 200,000 km:")
display(high_median_models)
```

Model groups with median mileage above 200,000 km:

	Brand	model	model_median_km
64	Force	Motors FM Force One Test	385000.0
305	Mitsubishi	Lancer	225000.0
198	Mahindra	Verito	220000.0

54. Inspect Extreme Mileage Records

Some Brand and model groups have unusually low or high median mileage.

A low median may be legitimate for a new or demo vehicle, while a very high median may represent heavy usage or possible data-quality issues.

Before using these medians for missing-value imputation, we will inspect the original mileage records belonging to these groups.

No records will be removed in this section.

```
# Combine the extreme groups we want to investigate
extreme_models = pd.concat(
    [
        low_median_models,
        high_median_models
    ]
).drop_duplicates(
    subset=["Brand", "model"]
)

print("Extreme Brand + model groups to inspect:")
display(extreme_models)
```

Extreme Brand + model groups to inspect:

	Brand	model	model_median_km
1	Ashok	Leyland Stile	0.0
145	Kia	CERATO	0.0
197	Mahindra	Thar Roxx	0.0
339	Rolls-Royce	Phantom Drop HeadCoupe	2.0
175	Mahindra	Imperio	10.0
174	Mahindra	Five-door Thar	45.0
372	Tata	Punch EV	225.0
13	Audi	Q3 Sportback	1100.0
291	Mercedes-Benz	SL-Class	1400.0
233	Maruti Suzuki	Jimny	1800.0
201	Mahindra	XUV 3XO	2500.0
64	Force	Motors FM Force One Test	385000.0
305	Mitsubishi	Lancer	225000.0
198	Mahindra	Verito	220000.0

```
# Inspect the actual mileage records for these extreme groups

extreme_records = known_km_records.merge(
    extreme_models[["Brand", "model"]],
    on=["Brand", "model"],
    how="inner"
)

extreme_records = extreme_records[
    [
        "Brand",
```

```

        "model",
        "Year",
        "Age",
        "kmDriven",
        "kmDriven_numeric",
        "AskPrice"
    ]
}.sort_values(
    ["Brand", "model", "kmDriven_numeric"]
)

print("Underlying records for extreme mileage groups:")
display(extreme_records)

```

Underlying records for extreme mileage groups:

	Brand	model	Year	Age	kmDriven	kmDriven_numeric	AskPrice
13	Ashok	Leyland Stile	2024	0	0 km	0.0	₹ 32,000
25	Audi	Q3 Sportback	2013	11	0 km	0.0	₹ 95,00,000
17	Audi	Q3 Sportback	2023	1	2,200 km	2200.0	₹ 1,00,000
6	Force	Motors FM Force One Test	2016	8	70,000 km	70000.0	₹ 5,60,000
14	Force	Motors FM Force One Test	2016	8	700,000 km	700000.0	₹ 5,60,000
16	Kia	CERATO	2024	0	0 km	0.0	₹ 45,000
19	Mahindra	Five-door Thar	2024	0	45 km	45.0	₹ 25,50,000
23	Mahindra	Imperio	2014	10	10 km	10.0	₹ 3,60,000
18	Mahindra	Thar Roxx	2024	0	0 km	0.0	₹ 21,000
24	Mahindra	Verito	2013	11	90,000 km	90000.0	₹ 3,40,000
2	Mahindra	Verito	2013	11	112,000 km	112000.0	₹ 3,60,000
0	Mahindra	Verito	2011	13	180,000 km	180000.0	₹ 1,99,000
5	Mahindra	Verito	2012	12	220,000 km	220000.0	₹ 2,40,000
9	Mahindra	Verito	2012	12	220,000 km	220000.0	₹ 2,40,000
11	Mahindra	Verito	2011	13	220,000 km	220000.0	₹ 2,35,000
15	Mahindra	Verito	2012	12	220,000 km	220000.0	₹ 2,40,000
21	Mahindra	XUV 3XO	2024	0	2,500 km	2500.0	₹ 14,70,000
22	Maruti Suzuki	Jimny	2023	1	500 km	500.0	₹ 6,00,000
10	Maruti Suzuki	Jimny	2023	1	521.0 km	521.0	₹ 14,35,000
1	Maruti Suzuki	Jimny	2023	1	1,800 km	1800.0	₹ 14,50,000
7	Maruti Suzuki	Jimny	2023	1	5,500 km	5500.0	₹ 9,55,000
3	Maruti Suzuki	Jimny	2024	0	11,000 km	11000.0	₹ 11,90,000
12	Mercedes-Benz	SL-Class	2024	0	1,400 km	1400.0	₹ 2,45,00,000
4	Mitsubishi	Lancer	1999	25	225,000 km	225000.0	₹ 90,000
8	Rolls-Royce	Phantom Drop HeadCoupe	2019	5	2 km	2.0	₹ 2,35,000
20	Tata	Punch EV	2024	0	225 km	225.0	₹ 65,000

55. Check Whether Extreme Mileage Groups Affect Missing Records

The previous analysis identified several Brand and model groups with unusually low or high mileage.

Before deciding how to impute missing kmDriven values, we will check whether any of the 88 missing-mileage records belong to these groups.

If an extreme group does not contain a missing-mileage record, its unusual values will not directly affect our imputation decision.

This allows us to avoid removing valid records unnecessarily.

```

# Check whether missing-mileage records belong to extreme Brand + model groups

extreme_missing_records = missing_km_records.merge(
    extreme_models[["Brand", "model"]],
    on=["Brand", "model"],
    how="inner"
)

```

```
print("Missing-mileage records belonging to extreme groups:")
print(len(extreme_missing_records))

display(
    extreme_missing_records[
        [
            "Brand",
            "model",
            "Year",
            "Age",
            "Transmission",
            "Owner",
            "FuelType",
            "AskPrice"
        ]
    ]
)
```

Missing-mileage records belonging to extreme groups:
0

Brand	model	Year	Age	Transmission	Owner	FuelType	AskPrice
-------	-------	------	-----	--------------	-------	----------	----------

56. Create Model-Level Mileage Imputation Values

The previous analysis showed that most missing-mileage records have other vehicles with the same Brand and model and known mileage.

We will therefore use the median mileage of each Brand + model group as the primary imputation value.

The median is preferred over the mean because mileage data can contain unusually high values.

For Brand + model groups without a known mileage value, we will use the overall median mileage as a fallback.

This gives every missing kmDriven value a reasonable numerical value while preserving the vehicle record.

```
# Calculate the overall median mileage from known values
overall_km_median = clean_df["kmDriven_numeric"].median()

# Create a lookup table containing the median mileage
# for each Brand + model combination
model_km_median_lookup = (
    known_km_records
    .groupby(["Brand", "model"])["kmDriven_numeric"]
    .median()
    .reset_index(name="model_median_km")
)

print("Overall median kmDriven:", overall_km_median)

print("\nNumber of Brand + model groups with a mileage median:")
print(len(model_km_median_lookup))

print("\nSample mileage lookup:")
display(model_km_median_lookup.head(10))
```

Overall median kmDriven: 65200.0

Number of Brand + model groups with a mileage median:
435

Sample mileage lookup:

	Brand	model	model_median_km
0	Ambassador	Ambassador	27009.0
1	Ashok	Leyland Stile	0.0
2	Aston Martin	Vanquish	25700.0
3	Audi	2.9 Sportback	44000.0
4	Audi	A3	42000.0
5	Audi	A3 Cabriolet	37909.0
6	Audi	A4	73000.0
7	Audi	A5	90000.0
8	Audi	A6	68988.5
9	Audi	A8	77665.0

57. Impute Missing kmDriven Values

We will now fill the missing `kmDriven` values.

The primary strategy is to use the median mileage of the same Brand + model.

If a Brand + model combination does not have any known mileage records, the overall dataset median will be used as a fallback.

The original `kmDriven` column is preserved, and the completed numerical column will be stored separately.

This allows us to compare the data before and after imputation.

```
# Start with the existing numerical mileage column
clean_df["kmDriven_imputed"] = clean_df["kmDriven_numeric"]

# Match each car with the median mileage of its Brand + model
clean_df = clean_df.merge(
    model_km_median_lookup,
    on=["Brand", "model"],
    how="left"
)

# Fill missing mileage using the Brand + model median
clean_df["kmDriven_imputed"] = clean_df["kmDriven_imputed"].fillna(
    clean_df["model_median_km"]
)

# If no model-level median exists, use the overall median
clean_df["kmDriven_imputed"] = clean_df["kmDriven_imputed"].fillna(
    overall_km_median
)

print("Missing values before/after imputation:")

print(
    "Before:",
    clean_df["kmDriven_numeric"].isnull().sum()
)

print(
    "After:",
    clean_df["kmDriven_imputed"].isnull().sum()
)

print("\nSample of imputed mileage values:")

display(
    clean_df[
        [
            "Brand",
            "model",
            "Year",
            "kmDriven",
            "kmDriven_numeric",
            "model_median_km",
            "kmDriven_imputed"
        ]
    ]
    .loc[clean_df["kmDriven_numeric"].isnull()]
    .head(20)
)
```

Missing values before/after imputation:
 Before: 88
 After: 0

Sample of imputed mileage values:

	Brand	model	Year	kmDriven	kmDriven_numeric	model_median_km	kmDriven_imputed
231	Maruti Suzuki	Alto-800	2019	NaN	NaN	71000.0	71000.0
353	Toyota	Innova	2016	NaN	NaN	157000.0	157000.0
444	Honda	City ZX	2024	NaN	NaN	87009.5	87009.5
476	Hyundai	Santro Xing	2010	NaN	NaN	81000.0	81000.0
608	Toyota	Etios	2012	NaN	NaN	87000.0	87000.0
793	Skoda	Kushaq	2010	NaN	NaN	21000.0	21000.0
976	Mahindra	Bolero Neo Plus	2015	NaN	NaN	100001.5	100001.5
1164	Mahindra	Bolero Neo	2023	NaN	NaN	42000.0	42000.0
1199	Tata	Indica	2014	NaN	NaN	107500.0	107500.0
1413	Hyundai	Santro Xing	2007	NaN	NaN	81000.0	81000.0
1978	Tata	Tiago Nrg	2021	NaN	NaN	35500.0	35500.0
2680	Maruti Suzuki	Zen-Estilo	2008	NaN	NaN	80250.0	80250.0
2847	Maruti Suzuki	S-Presso	2021	NaN	NaN	24008.0	24008.0
3009	Maruti Suzuki	Baleno	2018	NaN	NaN	54000.0	54000.0
3361	Ambassador	Ambassador	2008	NaN	NaN	27009.0	27009.0
3418	Honda	City	2001	NaN	NaN	72000.0	72000.0
3748	Mahindra	Bolero Power Plus	2015	NaN	NaN	144500.0	144500.0
3868	Maruti Suzuki	Esteem	2007	NaN	NaN	55000.0	55000.0
3921	Maruti Suzuki	Omni	2008	NaN	NaN	75000.0	75000.0
4191	ICML	Rhino Rx	2010	NaN	NaN	NaN	65200.0

58. Verify Imputed kmDriven Values

The missing mileage values have been filled using Brand + model medians, with the overall median as a fallback.

Before using the completed mileage feature for machine learning, we will verify that:

- No missing values remain.
- No negative mileage values were introduced.
- The imputed mileage column is numeric.
- The minimum and maximum values remain within the observed dataset range.

This is the final validation of the mileage-imputation step.

```
print("Data type:", clean_df["kmDriven_imputed"].dtype)

print(
    "Missing values:",
    clean_df["kmDriven_imputed"].isnull().sum()
)

print(
    "Negative values:",
    (clean_df["kmDriven_imputed"] < 0).sum()
)

print(
    "Minimum kmDriven:",
    clean_df["kmDriven_imputed"].min()
)

print(
    "Maximum kmDriven:",
    clean_df["kmDriven_imputed"].max()
)

print("\nSummary statistics:")
print(clean_df["kmDriven_imputed"].describe())
```

```
Data type: float64
Missing values: 0
Negative values: 0
Minimum kmDriven: 0.0
Maximum kmDriven: 980002.0

Summary statistics:
count      14988.000000
mean       71066.864191
std        57319.464288
min         0.000000
25%        44197.000000
50%        65321.000000
75%        86000.000000
max        980002.000000
Name: kmDriven_imputed, dtype: float64
```

59. Create the Final Clean Dataset

The missing-mileage analysis and validation are now complete.

We will create the final cleaned dataset for machine learning.

Temporary analysis columns such as `kmDriven_numeric` and `model_median_km` were useful during data cleaning but should not be included as model features.

The completed `kmDriven_imputed` column will replace the original `kmDriven` column.

The final dataset will therefore contain the original vehicle information with a complete numerical mileage feature.

```
# Create a copy so that the original cleaned dataframe remains available
final_df = clean_df.copy()

# Replace the original mileage column with the completed numerical version
final_df["kmDriven"] = final_df["kmDriven_imputed"]

# Remove temporary columns used only for analysis
final_df = final_df.drop(
    columns=[
        "kmDriven_numeric",
        "kmDriven_imputed",
        "model_median_km"
    ]
)

print("Final dataset shape:", final_df.shape)

print("\nFinal columns:")
print(final_df.columns.tolist())

print("\nMissing values:")
print(final_df.isnull().sum())
```

Final dataset shape: (14988, 11)

```
Final columns:
['Brand', 'model', 'Year', 'Age', 'kmDriven', 'Transmission', 'Owner', 'FuelType', 'PostedDate', 'AdditionInfo', 'AskPrice']

Missing values:
Brand      0
model      0
Year       0
Age        0
kmDriven   0
Transmission  0
Owner      0
FuelType   0
PostedDate 0
AdditionInfo 0
AskPrice   0
dtype: int64
```

60. Select Features for Machine Learning

The final cleaned dataset contains vehicle information and the target variable `AskPrice`.

For the machine-learning model, we will use features that describe the vehicle and can reasonably be provided by a user when requesting a price prediction.

The `AskPrice` column is the target variable and must not be included in the input features.

`PostedDate` and `AdditionInfo` will not be used in the baseline model because they are less structured and are not required for the basic prediction pipeline.

We will use the following features:

- Brand
- model
- Year
- Age
- kmDriven
- Transmission
- Owner
- FuelType

The target variable is `AskPrice`.

```
feature_columns = [
    "Brand",
    "model",
    "Year",
    "Age",
    "kmDriven",
    "Transmission",
    "Owner",
    "FuelType"
]

# Input features
X = final_df[feature_columns].copy()

# Convert AskPrice to numeric for machine learning
y = (
    final_df["AskPrice"]
    .astype(str)
    .str.replace("₹", "", regex=False)
    .str.replace(",", "", regex=False)
    .str.strip()
)

y = pd.to_numeric(y, errors="coerce")

print("Feature shape:", X.shape)
print("Target shape:", y.shape)

print("\nFeatures:")
print(X.columns.tolist())

print("\nTarget:")
print("AskPrice")

print("\nTarget data type:", y.dtype)
print("Missing target values:", y.isnull().sum())
```

```
Feature shape: (14988, 8)
Target shape: (14988,)
```

```
Features:
['Brand', 'model', 'Year', 'Age', 'kmDriven', 'Transmission', 'Owner', 'FuelType']
```

```
Target:
AskPrice
```

```
Target data type: int64
Missing target values: 0
```

61. Split Data into Training and Testing Sets

We will divide the dataset into training and testing subsets.

The training set will be used by the machine-learning model to learn patterns between vehicle characteristics and price.

The testing set will be kept separate and used to evaluate how well the trained model performs on unseen data.

We use an 80/20 split:

- 80% of the data → training
- 20% of the data → testing

The `random_state` is fixed so that the same split can be reproduced.

```

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("Training feature shape:", X_train.shape)
print("Testing feature shape:", X_test.shape)

print("Training target shape:", y_train.shape)
print("Testing target shape:", y_test.shape)

Training feature shape: (11990, 8)
Testing feature shape: (2998, 8)
Training target shape: (11990,)
Testing target shape: (2998,)

```

62. Prepare Categorical and Numerical Features

The dataset contains both categorical and numerical features.

Categorical features contain labels or categories such as Brand, model, Transmission, Owner, and FuelType.

Numerical features contain measurable values such as Year, Age, and kmDriven.

Machine-learning algorithms require numerical input, so categorical features will be converted using One-Hot Encoding.

We use a ColumnTransformer so that categorical features are encoded while numerical features are passed through unchanged.

The encoder uses `handle_unknown="ignore"` so that the model can safely process a category that was not present during training.

```

from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer

# Categorical features
categorical_columns = [
    "Brand",
    "model",
    "Transmission",
    "Owner",
    "FuelType"
]

# Numerical features
numerical_columns = [
    "Year",
    "Age",
    "kmDriven"
]

print("Categorical features:")
print(categorical_columns)

print("\nNumerical features:")
print(numerical_columns)

# Create preprocessing pipeline
preprocessor = ColumnTransformer(
    transformers=[
        (
            "categorical",
            OneHotEncoder(handle_unknown="ignore"),
            categorical_columns
        )
    ],
    remainder="passthrough"
)

print("\nPreprocessor created successfully.")

Categorical features:
['Brand', 'model', 'Transmission', 'Owner', 'FuelType']

Numerical features:
['Year', 'Age', 'kmDriven']

Preprocessor created successfully.

```


63. Fit and Transform the Data

The preprocessing pipeline must first learn the categories present in the training data.

We therefore use `fit_transform()` on the training data.

For the testing data, we use only `transform()`.

This is important because the test set must not influence the preprocessing stage.

The fitted preprocessor converts categorical features into numerical one-hot encoded columns while keeping the numerical features unchanged.

```
# Fit the preprocessor using training data
X_train_processed = preprocessor.fit_transform(X_train)

# Transform testing data using the already-fitted preprocessor
X_test_processed = preprocessor.transform(X_test)

print("Original training shape:", X_train.shape)
print("Processed training shape:", X_train_processed.shape)

print("\nOriginal testing shape:", X_test.shape)
print("Processed testing shape:", X_test_processed.shape)

Original training shape: (11990, 8)
Processed training shape: (11990, 465)

Original testing shape: (2998, 8)
Processed testing shape: (2998, 465)
```

64. Train the Random Forest Regression Model

We will use Random Forest Regression to predict used-car prices.

Random Forest is an ensemble machine-learning algorithm that combines predictions from multiple decision trees.

Each decision tree learns different patterns from the training data.

The final prediction is produced by combining the predictions of the individual trees.

We will use 100 trees and a fixed random state so that the experiment is reproducible.

```
from sklearn.ensemble import RandomForestRegressor

# Create the Random Forest regression model
model = RandomForestRegressor(
    n_estimators=100,
    random_state=42,
    n_jobs=-1
)

print("Training Random Forest model...")

# Train the model
model.fit(X_train_processed, y_train)

print("Model training completed successfully.")

Training Random Forest model...
Model training completed successfully.
```

65. Generate Predictions

The Random Forest model has now been trained using the training data.

We will now use the trained model to predict the prices of cars in the testing dataset.

The testing features have already been transformed using the same preprocessor that was fitted on the training data.

The model will produce one predicted price for each car in the testing dataset.

These predictions will later be compared with the actual prices to evaluate model performance.

```
# Generate predictions for the testing dataset
y_pred = model.predict(X_test_processed)

print("Number of predictions:", len(y_pred))
```

```
print("\nSample predicted prices:")
print(y_pred[:10])
```

Number of predictions: 2998

```
Sample predicted prices:
[ 347850.37238095  617030.          329850.          313212.12
 306091.42666667  6377300.         220659.99         3530310.
 424230.          227839.96         ]
```

66. Evaluate Model Performance

We will now evaluate how accurately the Random Forest model predicts used-car prices.

Since this is a regression problem, we will use three evaluation metrics:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- R^2 Score

MAE measures the average absolute difference between actual and predicted prices.

RMSE measures prediction error while giving more weight to larger errors.

R^2 Score measures how much of the variation in car prices is explained by the model.

These metrics allow us to quantitatively measure the performance of the trained model on unseen testing data.

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

# Calculate evaluation metrics
mae = mean_absolute_error(y_test, y_pred)

rmse = np.sqrt(mean_squared_error(y_test, y_pred))

r2 = r2_score(y_test, y_pred)

print("Model Evaluation Results")
print("-----")
print("MAE :", mae)
print("RMSE:", rmse)
print("R^2  :", r2)
```

```
Model Evaluation Results
-----
MAE : 158052.08408807102
RMSE: 507957.8073318212
R^2  : 0.8882930038018921
```

67. Compare Actual and Predicted Prices

We will compare the actual car prices with the prices predicted by the Random Forest model.

A good regression model should produce predictions close to the actual values.

We will use a scatter plot:

- X-axis → Actual price
- Y-axis → Predicted price
- Diagonal line → Perfect prediction

Points close to the diagonal line indicate accurate predictions.

Points far away from the diagonal line indicate larger prediction errors.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 6))

plt.scatter(y_test, y_pred, alpha=0.5)

# Perfect prediction line
min_price = min(y_test.min(), y_pred.min())
max_price = max(y_test.max(), y_pred.max())

plt.plot(
    [min_price, max_price],
```

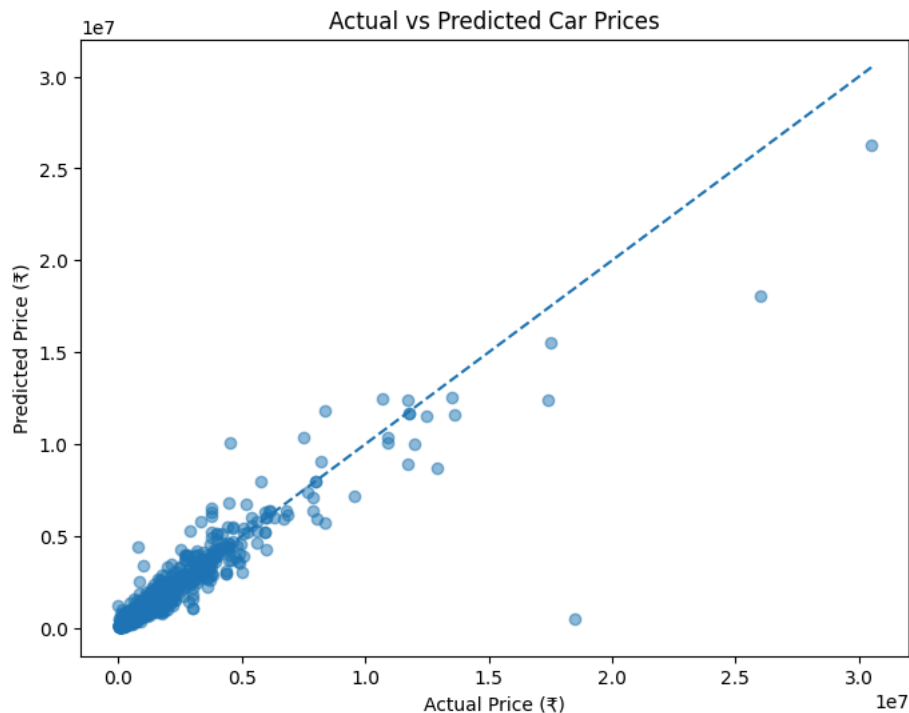
```

    [min_price, max_price],
    linestyle="--"
)

plt.xlabel("Actual Price (₹)")
plt.ylabel("Predicted Price (₹)")
plt.title("Actual vs Predicted Car Prices")

plt.show()

```



68. Investigate the Largest Prediction Errors

The evaluation metrics tell us how well the model performs overall.

However, they do not tell us which individual cars were difficult to predict.

We will calculate the prediction error for every test record and identify the cars with the largest errors.

This helps us understand:

- which cars are difficult for the model to predict
- whether expensive cars produce larger errors
- whether unusual cars or rare categories affect predictions
- whether the dataset contains possible outliers

The absolute error is calculated as:

Absolute Error = |Actual Price - Predicted Price|

```

import pandas as pd
import numpy as np

# Create a table containing actual and predicted prices
error_analysis = X_test.copy()

error_analysis["ActualPrice"] = y_test.values
error_analysis["PredictedPrice"] = y_pred

# Calculate prediction error
error_analysis["Error"] = (
    error_analysis["ActualPrice"] -
    error_analysis["PredictedPrice"]
)

# Calculate absolute error
error_analysis["AbsoluteError"] = (
    error_analysis["Error"].abs()
)

# Sort from largest error to smallest error

```

```

largest_errors = error_analysis.sort_values(
    "AbsoluteError",
    ascending=False
)

print("Top 20 largest prediction errors:")
display(
    largest_errors.head(20)
)

```

Top 20 largest prediction errors:

	Brand	model	Year	Age	kmDriven	Transmission	Owner	FuelType	ActualPrice	PredictedPrice	Er
7614	Bentley	Flying Spur	2017	7	22000.0	Automatic	first	Petrol	18500000	5.180800e+05	1.798192e
811	Mercedes-Benz	G Class	2019	5	59000.0	Automatic	first	Petrol	26000000	1.806095e+07	7.939050e
2955	Mercedes-Benz	GLA	2024	0	4700.0	Automatic	first	Petrol	4550000	1.006974e+07	-5.519740e
9494	Land Rover	Range Rover	2019	5	29700.0	Automatic	second	Diesel	17400000	1.240132e+07	4.998679e
10290	Mercedes-Benz	G Class	2024	0	3500.0	Automatic	first	Diesel	30500000	2.624820e+07	4.251800e
13335	Toyota	VELLFIRE	2024	0	500.0	Automatic	first	hybrid	12900000	8.709000e+06	4.191000e
8132	Toyota	Innova	2013	11	250000.0	Manual	second	Diesel	799999	4.398889e+06	-3.598890e
1858	BMW	7 Series	2021	3	18000.0	Automatic	first	Diesel	8400000	1.182311e+07	-3.423110e
3946	Toyota	Land Cruiser	2014	10	90000.0	Automatic	first	Diesel	7499000	1.037158e+07	-2.872580e
9248	Land Rover	Range Rover Sport	2019	5	71000.0	Automatic	second	Diesel	11700000	8.931500e+06	2.768500e
8457	Mercedes-Benz	A-class Limousine	2023	1	15000.0	Automatic	first	Diesel	3800000	6.515500e+06	-2.715500e
10069	BMW	M2	2020	4	9000.0	Automatic	first	Petrol	8400000	5.731881e+06	2.668119e
1634	BMW	2 Series	2022	2	14000.0	Automatic	first	Diesel	3800000	6.304900e+06	-2.504900e
11906	Toyota	2.8 Legend 4X2	2021	3	10000.0	Automatic	first	hybrid	1000000	3.434790e+06	-2.434790e

69. Analyze Prediction Errors by Price Range

The largest absolute errors are mostly associated with expensive vehicles.

We will divide the test data into different price ranges and calculate the average prediction error for each range.

This helps us understand whether the model performs differently for:

- Budget cars
- Mid-range cars
- Premium cars
- Luxury cars

Absolute error is useful for understanding the amount of money by which the prediction is wrong.

```

# Create a copy for price-range analysis
price_error_analysis = error_analysis.copy()

# Create price categories using the actual price
price_error_analysis["PriceRange"] = pd.cut(
    price_error_analysis["ActualPrice"],
    bins=[
        0,
        500000,
        1000000,
        2500000,
        5000000,
        float("inf")
    ],
    labels=[
        "Below ₹5 Lakh",
        "₹5-10 Lakh",
        "₹10-25 Lakh",

```

```

        "₹25-50 Lakh",
        "Above ₹50 Lakh"
    ]
)

# Calculate average error for each price range
price_range_summary = (
    price_error_analysis
    .groupby("PriceRange", observed=False)
    ["AbsoluteError"]
    .agg(["count", "mean", "median"])
    .reset_index()
)

print("Prediction error by actual price range:")

display(price_range_summary)

```

Prediction error by actual price range:

	PriceRange	count	mean	median
0	Below ₹5 Lakh	1301	6.192790e+04	40830.00000
1	₹5-10 Lakh	941	1.024240e+05	64449.99000
2	₹10-25 Lakh	550	2.151157e+05	145408.21125
3	₹25-50 Lakh	155	6.052214e+05	375700.49000
4	Above ₹50 Lakh	51	1.662118e+06	798250.00000

70. Analyze the Distribution of Car Prices

Car prices are not distributed uniformly.

Most used cars are expected to fall within the lower and middle price ranges, while luxury vehicles may have much higher prices.

We will visualize the distribution of `AskPrice` to understand the spread of the target variable.

This analysis helps us determine whether the target variable is highly skewed and whether expensive vehicles have a strong influence on the model.

```

import matplotlib.pyplot as plt

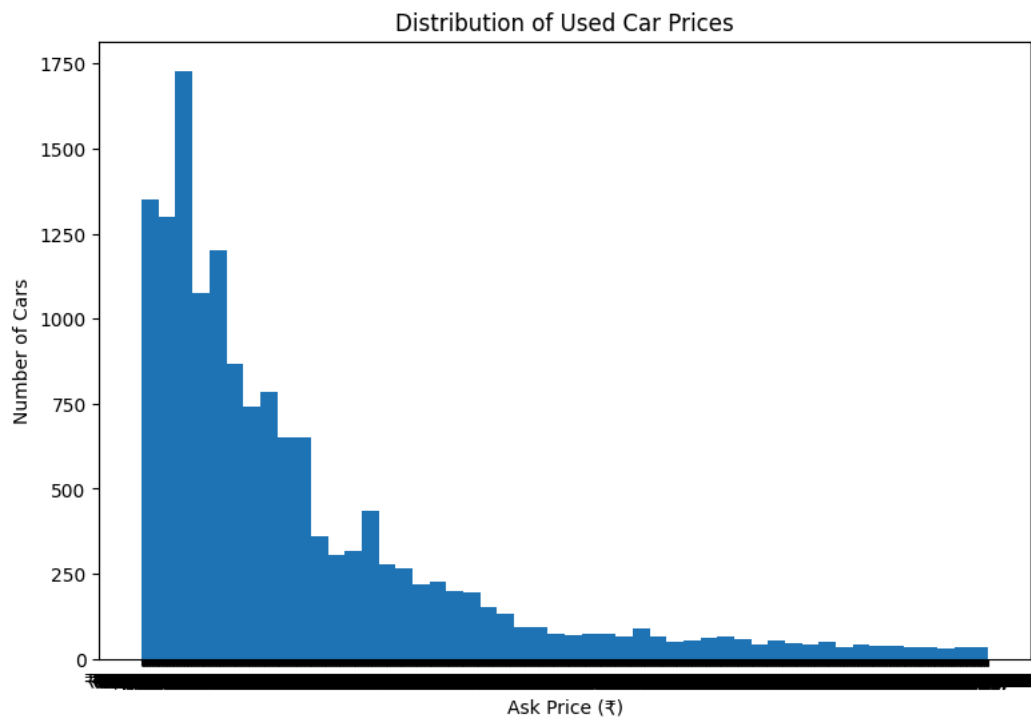
plt.figure(figsize=(9, 6))

plt.hist(
    final_df["AskPrice"],
    bins=50
)

plt.xlabel("Ask Price (₹)")
plt.ylabel("Number of Cars")
plt.title("Distribution of Used Car Prices")

plt.show()

```



71. Investigate Log Transformation of the Target Variable

The `AskPrice` target variable is strongly right-skewed.

A logarithmic transformation can reduce the effect of very large values and make a highly skewed distribution more balanced.

We will compare the original price distribution with its logarithmic representation.

This section is only an investigation. We will not replace the original target variable until we compare model performance.

```
import numpy as np
import matplotlib.pyplot as plt

# y is the cleaned numeric AskPrice target created in Section 60
log_prices = np.log1p(y)

print("Original price statistics:")
print(y.describe())

print("\nLog-transformed price statistics:")
print(log_prices.describe())

# Plot log-transformed prices
plt.figure(figsize=(9, 6))

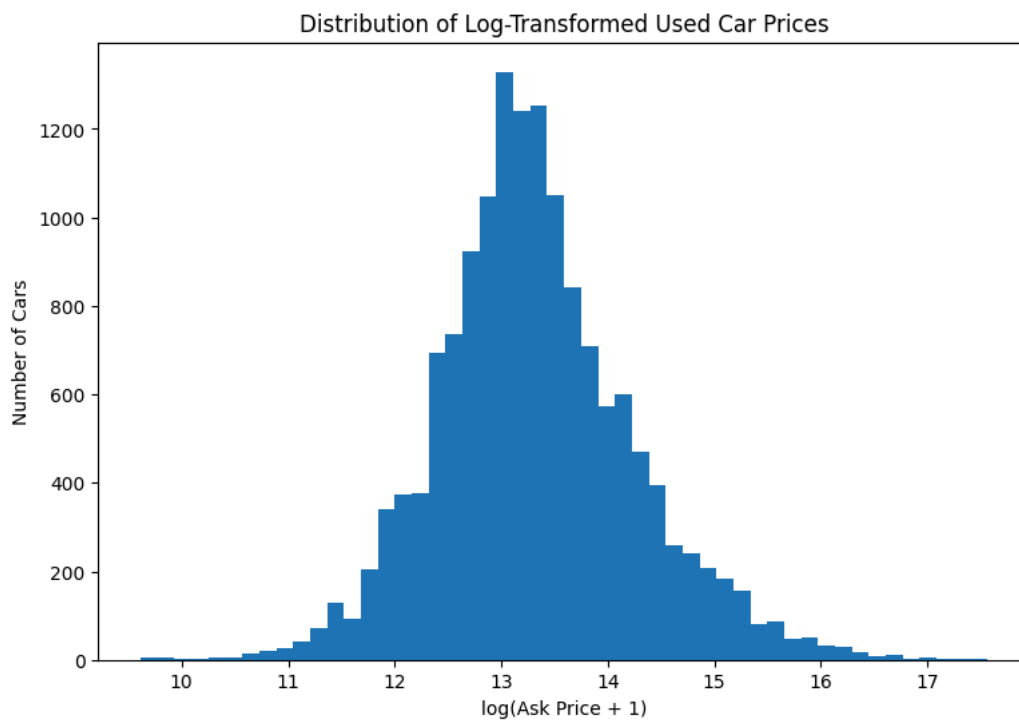
plt.hist(
    log_prices,
    bins=50
)

plt.xlabel("log(Ask Price + 1)")
plt.ylabel("Number of Cars")
plt.title("Distribution of Log-Transformed Used Car Prices")

plt.show()
```

```
Original price statistics:
count    1.498800e+04
mean      9.971514e+05
std       1.586859e+06
min       1.500000e+04
25%       3.490000e+05
50%       5.700000e+05
75%       1.025000e+06
max       4.250000e+07
Name: AskPrice, dtype: float64
```

```
Log-transformed price statistics:
count    14988.000000
mean      13.321163
std        0.919769
min        9.615872
25%       12.762830
50%       13.253393
75%       13.840204
max       17.565015
Name: AskPrice, dtype: float64
```



72. Inspect Kilometer Driven Distribution

Kilometers driven is an important numerical feature because vehicle usage can strongly affect used-car prices.

We will inspect its distribution to identify skewness and unusually high mileage values.

Understanding this distribution helps us identify potential data-quality issues before final model training.

```
import matplotlib.pyplot as plt

print("Kilometers Driven statistics:")
print(final_df["kmDriven"].describe())

print("\nMissing kilometers:")
print(final_df["kmDriven"].isnull().sum())

plt.figure(figsize=(9, 6))

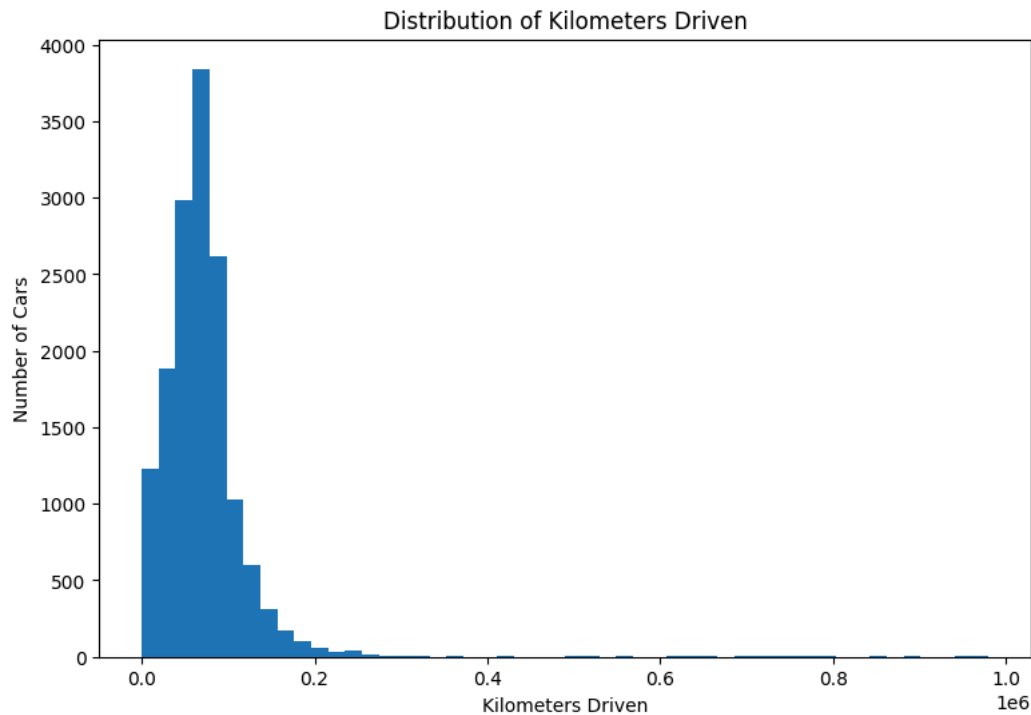
plt.hist(
    final_df["kmDriven"].dropna(),
    bins=50
)

plt.xlabel("Kilometers Driven")
plt.ylabel("Number of Cars")
plt.title("Distribution of Kilometers Driven")

plt.show()
```

```
Kilometers Driven statistics:
count    14988.000000
mean     71066.864191
std      57319.464288
min       0.000000
25%      44197.000000
50%      65321.000000
75%      86000.000000
max      980002.000000
Name: kmDriven, dtype: float64
```

```
Missing kilometers:
0
```



73. Investigate Extreme Kilometer Values

The kilometer distribution contains a long right tail with some unusually high mileage values.

Large values are not automatically invalid, so we will inspect the highest-mileage records before deciding whether they should be removed or retained.

This helps distinguish genuine high-mileage vehicles from possible data-entry or scraping errors.

```
# Display the 20 cars with the highest recorded mileage

highest_km = final_df.sort_values(
    by="kmDriven",
    ascending=False
)

print("Top 20 highest kilometer readings:")

display(
    highest_km[
        [
            "Brand",
            "model",
            "Year",
            "Age",
            "kmDriven",
            "Transmission",
            "Owner",
            "FuelType",
            "AskPrice"
        ]
    ].head(20)
)
```


Top 20 highest kilometer readings:

	Brand	model	Year	Age	kmDriven	Transmission	Owner	FuelType	AskPrice
6677	Volkswagen	Polo	2019	5	980002.0	Manual	first	Petrol	₹ 6,75,000
4767	Maruti Suzuki	Alto-800	2019	5	980000.0	Automatic	second	Hybrid/CNG	₹ 3,10,000
7737	Chevrolet	Cruze	2012	12	950000.0	Automatic	second	Diesel	₹ 3,50,000
13259	Hyundai	Accent	2011	13	950000.0	Automatic	first	Petrol	₹ 1,40,000
14906	Honda	Amaze	2018	6	950000.0	Manual	first	hybrid	₹ 3,20,000
11446	Maruti Suzuki	Swift-Dzire-Tour	2022	2	901500.0	Automatic	first	hybrid	₹ 6,90,000
13110	Maruti Suzuki	Eeco	2019	5	900000.0	Automatic	second	hybrid	₹ 3,45,000
2222	Mercedes-Benz	GLE Class	2016	8	850000.0	Automatic	first	Diesel	₹ 26,00,000
3267	Mitsubishi	Pajero Sport	2015	9	850000.0	Automatic	second	Diesel	₹ 8,75,000
9410	Honda	Amaze	2013	11	850000.0	Automatic	second	Hybrid/CNG	₹ 4,50,000
10889	Maruti Suzuki	Alto	2012	12	820002.0	Manual	second	hybrid	₹ 1,10,000
10668	Maruti Suzuki	S-Presso	2020	4	801270.0	Automatic	first	hybrid	₹ 4,65,000
8981	Maruti Suzuki	S-Presso	2020	4	801270.0	Automatic	first	Hybrid/CNG	₹ 4,65,000
10881	Maruti Suzuki	Wagon-R	2020	4	800000.0	Automatic	first	hybrid	₹ 5,35,000
9312	Maruti Suzuki	Wagon R	2020	4	800000.0	Automatic	first	Hybrid/CNG	₹ 5,35,000
5765	Honda	City	2008	16	800000.0	Automatic	second	Hybrid/CNG	₹ 1,50,000
595	Tata	Tigor	2018	6	800000.0	Manual	second	Petrol	₹ 4,85,000
14974	Hyundai	Eon	2012	12	781313.0	Manual	second	Petrol	₹ 1,55,000
11472	Hyundai	i10	2010	14	780000.0	Manual	second	hybrid	₹ 1,42,000
7276	Maruti Suzuki	Ertiga	2020	4	780000.0	Automatic	first	Hybrid/CNG	₹ 9,49,999

74. Quantify Extreme Kilometer Values

The previous section showed several unusually high kilometer readings.

Before removing any records, we will quantify how many vehicles have extremely high mileage.

We will inspect several thresholds to understand how rare these values are in the dataset.

```
# Define mileage thresholds for investigation

thresholds = [
    200000,
    300000,
    500000,
    700000,
    800000,
    900000
]

print("Extreme kilometer analysis:")
print("-" * 40)

for threshold in thresholds:
    count = (final_df["kmDriven"] > threshold).sum()

    print(
        f"Cars above {threshold:,} km: "
        f"{count} ({count / len(final_df) * 100:.2f}%) "
    )
```

Extreme kilometer analysis:

```
-----
Cars above 200,000 km: 183 (1.22%)
Cars above 300,000 km: 74 (0.49%)
Cars above 500,000 km: 55 (0.37%)
Cars above 700,000 km: 38 (0.25%)
Cars above 800,000 km: 13 (0.09%)
Cars above 900,000 km: 6 (0.04%)
```

75. Inspect Extremely High Mileage Records

Only a very small number of vehicles have more than 900,000 kilometers recorded.

We will inspect these records individually to determine whether they appear to be genuine high-mileage vehicles or possible data-quality anomalies.

```
# Inspect vehicles with more than 900,000 km

extreme_km = final_df[
    final_df["kmDriven"] > 900000
].copy()

print("Number of vehicles above 900,000 km:")
print(len(extreme_km))

print("\nExtremely high mileage records:")

display(
    extreme_km[
        [
            "Brand",
            "model",
            "Year",
            "Age",
            "kmDriven",
            "Transmission",
            "Owner",
            "FuelType",
            "AskPrice"
        ]
    ].sort_values(
        by="kmDriven",
        ascending=False
    )
)
```

Number of vehicles above 900,000 km:
6

Extremely high mileage records:

	Brand	model	Year	Age	kmDriven	Transmission	Owner	FuelType	AskPrice
6677	Volkswagen	Polo	2019	5	980002.0	Manual	first	Petrol	₹ 6,75,000
4767	Maruti Suzuki	Alto-800	2019	5	980000.0	Automatic	second	Hybrid/CNG	₹ 3,10,000
7737	Chevrolet	Cruze	2012	12	950000.0	Automatic	second	Diesel	₹ 3,50,000
13259	Hyundai	Accent	2011	13	950000.0	Automatic	first	Petrol	₹ 1,40,000
14906	Honda	Amaze	2018	6	950000.0	Manual	first	hybrid	₹ 3,20,000
11446	Maruti Suzuki	Swift-Dzire-Tour	2022	2	901500.0	Automatic	first	hybrid	₹ 6,90,000

76. Calculate Approximate Annual Kilometers

Extremely high mileage should be interpreted together with vehicle age.

A useful diagnostic is to calculate approximate kilometers driven per year:

Annual Kilometers = kmDriven / Age

This helps identify records where the reported mileage is unusually high relative to the vehicle's age.

This analysis is used only for investigation at this stage. We will not automatically remove records based on this value.

```
# Calculate approximate annual kilometers

extreme_km_analysis = extreme_km.copy()

# Avoid division by zero
extreme_km_analysis["AnnualKm"] = (
    extreme_km_analysis["kmDriven"] /
    extreme_km_analysis["Age"].replace(0, np.nan)
)

display(
    extreme_km_analysis[
        [
            "Brand",
            "model",
            "Year",
            "Age",
            "kmDriven",
            "AnnualKm"
        ]
    ]
)
```

```

        "AnnualKm",
        "Transmission",
        "Owner",
        "FuelType",
        "AskPrice"
    ]
    ].sort_values(
        by="AnnualKm",
        ascending=False
    )
)

```

	Brand	model	Year	Age	kmDriven	AnnualKm	Transmission	Owner	FuelType	AskPrice
11446	Maruti Suzuki	Swift-Dzire-Tour	2022	2	901500.0	450750.000000	Automatic	first	hybrid	₹ 6,90,000
6677	Volkswagen	Polo	2019	5	980002.0	196000.400000	Manual	first	Petrol	₹ 6,75,000
4767	Maruti Suzuki	Alto-800	2019	5	980000.0	196000.000000	Automatic	second	Hybrid/CNG	₹ 3,10,000
14906	Honda	Amaze	2018	6	950000.0	158333.333333	Manual	first	hybrid	₹ 3,20,000
7737	Chevrolet	Cruze	2012	12	950000.0	79166.666667	Automatic	second	Diesel	₹ 3,50,000
13259	Hyundai	Accent	2011	13	950000.0	73076.923077	Automatic	first	Petrol	₹ 1,40,000

77. Identify Candidate Extreme-Mileage Records

The previous analysis showed that a small number of vehicles have extremely high reported mileage.

We will now create a candidate dataset excluding vehicles with more than 500,000 km.

This threshold is used for experimentation rather than being treated as an absolute rule.

We will compare model performance before and after this cleaning step.

The original dataset will be preserved so that the effect of the cleaning decision can be measured.

```

# Create a candidate dataset with extreme mileage removed

cleaned_km_df = final_df[
    final_df["kmDriven"] <= 500000
].copy()

removed_km_records = len(final_df) - len(cleaned_km_df)

print("Original records:", len(final_df))
print("Records after mileage filtering:", len(cleaned_km_df))
print("Records removed:", removed_km_records)
print(
    "Percentage removed:",
    f"{(removed_km_records / len(final_df)) * 100:.2f}%"
)

```

```

Original records: 14988
Records after mileage filtering: 14933
Records removed: 55
Percentage removed: 0.37%

```

78. Inspect Records Removed by Mileage Filtering

Before accepting the mileage filtering rule, we will inspect the records that would be removed.

This helps verify that the filtering rule affects only extreme mileage observations.

The removed records will not be permanently deleted from the original dataset.

```

# Inspect records that would be removed

removed_km_df = final_df[
    final_df["kmDriven"] > 500000
].copy()

print("Records above 500,000 km:", len(removed_km_df))

display(
    removed_km_df[
        [
            "Brand",
            "model",

```

```
        "Year",  
        "Age",  
        "kmDriven",  
        "Transmission",  
        "Owner",  
        "FuelType",  
        "AskPrice"  
    ]  
].sort_values(  
    by="kmDriven",  
    ascending=False  
)  
)
```


Records above 500,000 km: 55

	Brand	model	Year	Age	kmDriven	Transmission	Owner	FuelType	AskPrice
6677	Volkswagen	Polo	2019	5	980002.0	Manual	first	Petrol	₹ 6,75,000
4767	Maruti Suzuki	Alto-800	2019	5	980000.0	Automatic	second	Hybrid/CNG	₹ 3,10,000
7737	Chevrolet	Cruze	2012	12	950000.0	Automatic	second	Diesel	₹ 3,50,000
13259	Hyundai	Accent	2011	13	950000.0	Automatic	first	Petrol	₹ 1,40,000
14906	Honda	Amaze	2018	6	950000.0	Manual	first	hybrid	₹ 3,20,000

11446	Maruti Suzuki	Swift-Dzire-Tour	2022	2	901500.0	Automatic	first	hybrid	₹ 6,90,000
13110	Maruti Suzuki	Eeco	2019	5	900000.0	Automatic	second	hybrid	₹ 3,45,000

2222	Mercedes-Benz	GLE Class	2016	8	850000.0	Automatic	first	Diesel	₹ 26,00,000
------	---------------	-----------	------	---	----------	-----------	-------	--------	-------------

We previously identified 55 vehicles with reported mileage above 500,000 km.

Instead of permanently deleting these records, we will test whether removing extreme-mileage observations from the training data improves model performance.

9410	Honda	Amaze	2013	11	850000.0	Automatic	second	Hybrid/CNG	₹ 4,50,000
3267	Mitsubishi	Pajero Sport	2015	9	850000.0	Automatic	second	Diesel	₹ 8,75,000

The original test set will remain unchanged for both models.

10889	Maruti Suzuki	Alto	2012	12	820002.0	Manual	second	hybrid	₹ 1,10,000
8981	Maruti Suzuki	S-Presso	2020	4	801270.0	Automatic	first	Hybrid/CNG	₹ 4,65,000

This creates a controlled experiment because both models are evaluated on exactly the same unseen test data.

10668	Maruti Suzuki	S-Presso	2020	4	801270.0	Automatic	first	hybrid	₹ 4,65,000
-------	---------------	----------	------	---	----------	-----------	-------	--------	------------

We will compare:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- R-Score

10881	Maruti Suzuki	Wagon-R	2020	4	800000.0	Automatic	first	hybrid	₹ 5,35,000
8765	Honda	City	2008	16	800000.0	Automatic	second	Hybrid/CNG	₹ 1,50,000

The original Random Forest model is treated as the baseline.

595	Tata	Tigor	2018	6	800000.0	Manual	second	Petrol	₹ 4,85,000
14974	Hyundai	Eon	2012	12	781313.0	Manual	second	Petrol	₹ 1,55,000

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
```

```
# -----
# 1. Identify extreme-mileage records in training data
# -----
```

```
training_extreme_mask = X_train["kmDriven"] > 500000
```

```
print("Extreme-mileage records in training set:",
      training_extreme_mask.sum())
```

```
# Keep only records with kmDriven <= 500,000
X_train_clean = X_train.loc[~training_extreme_mask].copy()
y_train_clean = y_train.loc[~training_extreme_mask].copy()
```

```
print("Original training records:", len(X_train))
print("Cleaned training records:", len(X_train_clean))
print("Records removed from training:", len(X_train) - len(X_train_clean))
```

```
# -----
# 2. Create a NEW preprocessing pipeline
# -----
```

```
clean_preprocessor = ColumnTransformer(
    transformers=[
        (
            "categorical",
            OneHotEncoder(handle_unknown="ignore"),
            categorical_columns
        )
    ],
    remainder="passthrough"
)
```

```
# -----
# 3. Fit preprocessing ONLY on cleaned training data
# -----
```

```
X_train_clean_processed = clean_preprocessor.fit_transform(
    X_train_clean
)
```

```
X_test_clean_processed = clean_preprocessor.transform(
```

```

    X_test
)

print("\nCleaned training processed shape:",
      X_train_clean_processed.shape)

print("Test processed shape:",
      X_test_clean_processed.shape)

# -----
# 4. Train a second Random Forest model
# -----

clean_model = RandomForestRegressor(
    n_estimators=100,
    random_state=42,
    n_jobs=-1
)

print("\nTraining cleaned-data Random Forest...")

clean_model.fit(
    X_train_clean_processed,
    y_train_clean
)

print("Cleaned-data model training completed.")

# -----
# 5. Generate predictions on the SAME test set
# -----

clean_y_pred = clean_model.predict(
    X_test_clean_processed
)

# -----
# 6. Evaluate cleaned model
# -----

clean_mae = mean_absolute_error(
    y_test,
    clean_y_pred
)

clean_rmse = np.sqrt(
    mean_squared_error(
        y_test,
        clean_y_pred
    )
)

clean_r2 = r2_score(
    y_test,
    clean_y_pred
)

# -----
# 7. Display results
# -----

print("\nCleaned Model Evaluation Results")
print("-----")
print("MAE :", clean_mae)
print("RMSE:", clean_rmse)
print("R²  :", clean_r2)

```

```

Extreme-mileage records in training set: 40
Original training records: 11990
Cleaned training records: 11950
Records removed from training: 40

Cleaned training processed shape: (11950, 465)
Test processed shape: (2998, 465)

Training cleaned-data Random Forest...
Cleaned-data model training completed.

Cleaned Model Evaluation Results

```

```
-----
MAE : 161325.21829837537
RMSE: 524061.37241052033
R2 : 0.8810979359236942
```

80. Compare Baseline and Cleaned Model

The baseline Random Forest model and the cleaned-data Random Forest model were evaluated on the same test dataset.

The cleaned model was trained after removing training records with mileage above 500,000 km.

We will now compare the evaluation metrics and calculate the percentage change.

A lower MAE and RMSE indicate lower prediction error, while a higher R^2 indicates better explanatory performance.

The cleaning decision will be based on measured model performance rather than assuming that extreme values are always harmful.

```
# Store baseline and cleaned model metrics

comparison = {
    "Metric": ["MAE", "RMSE", "R2"],
    "Baseline Model": [
        mae,
        rmse,
        r2
    ],
    "Cleaned Model": [
        clean_mae,
        clean_rmse,
        clean_r2
    ]
}

comparison_df = pd.DataFrame(comparison)

# Calculate percentage change
comparison_df["Percentage Change"] = (
    (comparison_df["Cleaned Model"] - comparison_df["Baseline Model"])
    / comparison_df["Baseline Model"]
) * 100

print("Model Performance Comparison")
print("-----")

display(comparison_df)
```

```
Model Performance Comparison
-----
```

	Metric	Baseline Model	Cleaned Model	Percentage Change
0	MAE	158052.084088	161325.218298	2.070921
1	RMSE	507957.807332	524061.372411	3.170256
2	R ²	0.888293	0.881098	-0.809988

81. Investigate Extreme Mileage Records

The previous experiment showed that automatically removing vehicles above 500,000 km reduced model performance.

Therefore, these records will not be removed solely because of their high mileage.

Instead, we will investigate their relationship with vehicle age and selling price.

This helps distinguish unusual but potentially useful observations from clearly suspicious data records.

The purpose of this section is investigation rather than automatic deletion.

```
# Investigate extreme-mileage vehicles

extreme_mileage = final_df[
    final_df["kmDriven"] > 500000
].copy()

print("Extreme-mileage records:", len(extreme_mileage))

display(
    extreme_mileage[
        [
            "Brand",
```



```
        "model",
        "Year",
        "Age",
        "kmDriven",
        "Transmission",
        "Owner",
        "FuelType",
        "AskPrice"
    ]
    .sort_values("kmDriven", ascending=False)
)
```


Extreme-mileage records: 55

	Brand	model	Year	Age	kmDriven	Transmission	Owner	FuelType	AskPrice
6677	Volkswagen	Polo	2019	5	980002.0	Manual	first	Petrol	₹ 6,75,000
4767	Maruti Suzuki	Alto-800	2019	5	980000.0	Automatic	second	Hybrid/CNG	₹ 3,10,000
7737	Chevrolet	Cruze	2012	12	950000.0	Automatic	second	Diesel	₹ 3,50,000
13259	Hyundai	Accent	2011	13	950000.0	Automatic	first	Petrol	₹ 1,40,000
14906	Honda	Amaze	2018	6	950000.0	Manual	first	hybrid	₹ 3,20,000

11446	Maruti Suzuki	Swift-Dzire-Tour	2022	2	901500.0	Automatic	first	hybrid	₹ 6,90,000
-------	---------------	------------------	------	---	----------	-----------	-------	--------	------------

82. Calculate Mileage per Year

13110	Maruti Suzuki	Eeco	2019	5	900000.0	Automatic	second	hybrid	₹ 3,45,000
-------	---------------	------	------	---	----------	-----------	--------	--------	------------

2222	Mercedes-Benz	GLE Class	2016	8	850000.0	Automatic	first	Diesel	₹ 26,00,000
------	---------------	-----------	------	---	----------	-----------	-------	--------	-------------

A high total mileage may be reasonable for a heavily used vehicle.

9410	Honda	Amaze	2013	11	850000.0	Automatic	second	Hybrid/CNG	₹ 4,50,000
------	-------	-------	------	----	----------	-----------	--------	------------	------------

To investigate this further, we calculate approximate annual mileage using:

3267	Mitsubishi	Pajero Sport	2015	9	850000.0	Automatic	second	Diesel	₹ 8,75,000
------	------------	--------------	------	---	----------	-----------	--------	--------	------------

Annual Mileage = Total Kilometers Driven / Vehicle Age

10889	Maruti Suzuki	Alto	2012	12	820002.0	Manual	second	hybrid	₹ 1,10,000
-------	---------------	------	------	----	----------	--------	--------	--------	------------

This provides additional context for extreme-mileage observations.

8981	Maruti Suzuki	S-Presso	2020	4	801270.0	Automatic	first	Hybrid/CNG	₹ 4,65,000
------	---------------	----------	------	---	----------	-----------	-------	------------	------------

This value is used for investigation only and is not automatically used as a model feature.

10668	Maruti Suzuki	S-Presso	2020	4	801270.0	Automatic	first	hybrid	₹ 4,65,000
-------	---------------	----------	------	---	----------	-----------	-------	--------	------------

```
# Calculate approximate annual mileage

extreme_mileage_analysis = extreme_mileage.copy()

# Avoid division by zero
extreme_mileage_analysis["AnnualKm"] = (
    extreme_mileage_analysis["kmDriven"]
    / extreme_mileage_analysis["Age"].replace(0, np.nan)
)

display(
    extreme_mileage_analysis[
        [
            "Brand",
            "model",
            "Year",
            "Age",
            "kmDriven",
            "AnnualKm",
            "AskPrice"
        ]
    ]
    .sort_values("AnnualKm", ascending=False)
)
```

14173	Maruti Suzuki	Wagon-R-1-0	2012	12	750040.0	Manual	second	hybrid	₹ 2,20,000
2166	Maruti Suzuki	Swift-Dzire	2020	4	750000.0	Manual	first	Diesel	₹ 5,20,000
1704	Kia	Sonet	2021	3	730000.0	Automatic	second	Petrol	₹ 9,30,000
3724	Kia	Sonet	2021	3	730000.0	Automatic	second	Petrol	₹ 9,30,000
5769	Honda	City	2015	9	725000.0	Automatic	second	Petrol	₹ 4,50,000
7233	Maruti Suzuki	Eeco	2014	10	720000.0	Manual	first	Hybrid/CNG	₹ 3,50,000
14226	Maruti Suzuki	Estilo	2010	14	712000.0	Manual	first	hybrid	₹ 1,25,000
3983	Mahindra	Scorpio	2017	7	710205.0	Manual	first	Diesel	₹ 8,75,000
7223	Maruti Suzuki	Celerio	2017	7	710000.0	Automatic	first	Petrol	₹ 4,25,000
13528	Maruti Suzuki	Omni	2016	8	700311.0	Manual	first	hybrid	₹ 1,90,000
3463	Maruti Suzuki	Alto-800	2017	7	700000.0	Automatic	second	Petrol	₹ 2,00,000
8165	Force Motors FM	Force One Test	2016	8	700000.0	Automatic	first	Hybrid/CNG	₹ 5,60,000
5800	Honda	City	2014	10	680000.0	Automatic	second	Petrol	₹ 3,60,000
5330	Honda	WRV	2020	4	650000.0	Manual	first	Diesel	₹ 49,000
5556	Honda	WRV	2020	4	650000.0	Manual	first	Diesel	₹ 49,000
9590	Maruti Suzuki	Ertiga	2022	2	630000.0	Manual	second	hybrid	₹ 12,00,000
9940	Toyota	Etios Liva	2019	5	630000.0	Manual	second	Petrol	₹ 5,99,999
6457	Maruti Suzuki	Swift	2010	14	620287.0	Automatic	first	Hybrid/CNG	₹ 86,000
10075	Maruti Suzuki	Alto	2009	15	610679.0	Manual	first	hybrid	₹ 1,10,000
249	Maruti Suzuki	Ciaz	2020	4	610003.0	Manual	first	Petrol	₹ 8,40,000

893	Ford	Ecosport	2016	8	610000.0	Manual	first	Diesel	₹ 4,90,000
14097	Maruti Suzuki	Swift-Dzire	2012	12	570000.0	Automatic	second	Petrol	₹ 3,75,000
10629	Maruti Suzuki	Vitara-Brezza	2018	6	550002.0	Automatic	second	Diesel	₹ 7,90,000
14054	Renault	KWID	2020	4	550000.0	Automatic	first	Petrol	₹ 4,15,000
10646	Hyundai	i10	2011	13	530000.0	Manual	second	hybrid	₹ 1,90,000
13574	Maruti Suzuki	Swift-Dzire	2018	6	522168.0	Automatic	first	hybrid	₹ 2,58,000
2369	Maruti Suzuki	Ignis	2019	5	520000.0	Manual	first	Petrol	₹ 4,50,000

	Brand	model	Year	Age	kmDriven	AnnualKm	AskPrice
11446	Maruti Suzuki	Swift-Dzire-Tour	2022	2	901500.0	450750.000000	₹ 6,90,000
13184	Maruti Suzuki	Swift-Dzire	2022	2	780000.0	390000.000000	₹ 7,60,000
9590	Maruti Suzuki	Ertiga	2022	2	630000.0	315000.000000	₹ 12,00,000
3724	Kia	Sonet	2021	3	730000.0	243333.333333	₹ 9,30,000
1704	Kia	Sonet	2021	3	730000.0	243333.333333	₹ 9,30,000

8981	Maruti Suzuki	S-Presso	2020	4	801270.0	200317.500000	₹ 4,65,000
10668	Maruti Suzuki	S-Presso	2020	4	801270.0	200317.500000	₹ 4,65,000
10881	Maruti Suzuki	Wagon R	2020	4	800000.0	200000.000000	₹ 5,35,000
9312	Maruti Suzuki	Wagon R	2020	4	800000.0	200000.000000	₹ 5,35,000
6677	Volkswagen	Polo	2019	5	980002.0	196000.400000	₹ 6,75,000
4767	Maruti Suzuki	Alto-800	2019	5	980000.0	196000.000000	₹ 3,10,000
7276	Maruti Suzuki	Ertiga	2020	4	780000.0	195000.000000	₹ 9,49,999

83. Identify Extremely High Annual Mileage

The annual mileage analysis shows that some vehicles have unusually high estimated yearly usage.

We will now count vehicles above several annual-mileage thresholds.

This is an investigation step only. No records will be deleted at this stage.

Using multiple thresholds helps us understand whether the unusual values are isolated records or a broader pattern in the dataset.

```
# Define annual mileage thresholds for investigation

annual_km_thresholds = [
    50000,
    75000,
    100000,
    150000,
    200000,
    300000
]

print("Annual mileage analysis")
print("-" * 30)

for threshold in annual_km_thresholds:
    count = (extreme_mileage_analysis["AnnualKm"] > threshold).sum()

    print(
        f"Cars above {threshold:,} km/year: "
        f"{count} "
        f"({count / len(extreme_mileage_analysis) * 100:.2f}%)"
    )
```

13981	Maruti Suzuki	Scorpio	2017	7	710205.0	101457.857143	₹ 8,75,000
7223	Maruti Suzuki	Scorpio	2017	7	710000.0	101428.571429	₹ 4,25,000
3463	Maruti Suzuki	Alto-800	2017	7	700000.0	100000.000000	₹ 2,00,000
3267	Maruti Suzuki	Paldeo Sport	2015	9	850000.0	94444.444444	₹ 8,75,000
10629	Maruti Suzuki	Wagon R	2018	6	550002.0	91667.000000	₹ 7,90,000

13528	Maruti Suzuki	Omni	2016	8	700311.0	87538.875000	₹ 1,90,000
8185	Force Motors	Force One	2016	8	700000.0	87500.000000	₹ 5,60,000
13574	Maruti Suzuki	Swift-Dzire	2018	6	522168.0	87028.000000	₹ 2,58,000
5131	Hyundai	Grand i10	2015	9	770000.0	85555.555556	₹ 4,90,000
634	Renault	Duster	2015	9	760045.0	84449.444444	₹ 1,55,000
5769	Honda	City	2015	9	725000.0	80555.555556	₹ 4,50,000
7737	Chevrolet	Cruze	2012	12	950000.0	79166.666667	₹ 3,50,000

84. Inspect the Most Extreme Annual-Mileage Records

We will now inspect the records with the highest estimated annual mileage.

These records will be reviewed for combinations of vehicle age, model, mileage, fuel type, and price that may indicate possible data-quality problems.

No records will be removed based only on this analysis.

```
# Display the 20 records with the highest annual mileage

display(
    extreme_mileage_analysis[
        [
            "Brand",
            "model",
            "Year",
            "Age",
            "kmDriven",
            "AnnualKm",
            "Transmission",
            "Owner",
            "FuelType",
            "AskPrice"
        ]
    ]
)
```

```
.sort_values("AnnualKm", ascending=False)
.head(20)
)
```

14097	Maruti Suzuki	model	Year	Age	kmDriven	AnnualKm	Transmission	Owner	FuelType	AskPrice	
11446	Maruti Suzuki	Swift-Dzire-Tour	2022	2	901500.0	450750.000000	Automatic	first	hybrid	₹ 6,90,000	
10646	Hyundai	Swift-Dzire	2022	10	2011	530000.0	40769.230769	Manual	first	hybrid	₹ 1,90,000
13184	Maruti Suzuki	Swift-Dzire	2022	2	780000.0	390000.000000	Manual	first	hybrid	₹ 7,60,000	
9590	Maruti Suzuki	Ertiga	2022	2	630000.0	315000.000000	Manual	second	hybrid	₹ 12,00,000	
3724	Kia	Sonet	2021	3	730000.0	243333.333333	Automatic	second	Petrol	₹ 9,30,000	
1704	Kia	Sonet	2021	3	730000.0	243333.333333	Automatic	second	Petrol	₹ 9,30,000	
8981	Maruti Suzuki	S-Presso	2020	4	801270.0	200317.500000	Automatic	first	Hybrid/CNG	₹ 4,65,000	
10668	Maruti Suzuki	S-Presso	2020	4	801270.0	200317.500000	Automatic	first	hybrid	₹ 4,65,000	
10881	Maruti Suzuki	Wagon-R	2020	4	800000.0	200000.000000	Automatic	first	hybrid	₹ 5,35,000	
9312	Maruti Suzuki	Wagon R	2020	4	800000.0	200000.000000	Automatic	first	Hybrid/CNG	₹ 5,35,000	
6677	Volkswagen	Polo	2019	5	980002.0	196000.400000	Manual	first	Petrol	₹ 6,75,000	
4767	Maruti Suzuki	Alto-800	2019	5	980000.0	196000.000000	Automatic	second	Hybrid/CNG	₹ 3,10,000	
7276	Maruti Suzuki	Ertiga	2020	4	780000.0	195000.000000	Automatic	first	Hybrid/CNG	₹ 9,49,999	
1387	Tata	Nexon	2020	4	775000.0	193750.000000	Manual	first	Diesel	₹ 8,95,000	
2166	Maruti Suzuki	Swift-Dzire	2020	4	750000.0	187500.000000	Manual	first	Diesel	₹ 5,20,000	
13110	Maruti Suzuki	Eeco	2019	5	900000.0	180000.000000	Automatic	second	hybrid	₹ 3,45,000	
5330	Honda	WRV	2020	4	650000.0	162500.000000	Manual	first	Diesel	₹ 49,000	
5556	Honda	WRV	2020	4	650000.0	162500.000000	Manual	first	Diesel	₹ 49,000	
14906	Honda	Amaze	2018	6	950000.0	158333.333333	Manual	first	hybrid	₹ 3,20,000	
249	Maruti Suzuki	Ciaz	2020	4	610003.0	152500.750000	Manual	first	Petrol	₹ 8,40,000	
5423	Maruti Suzuki	Eeco	2019	5	760000.0	152000.000000	Manual	second	Hybrid/CNG	₹ 2,90,000	

85. Investigate Duplicate Vehicle Records

The extreme-mileage analysis revealed that some records appear to have identical vehicle information and price.

Before deciding whether extreme mileage values should be removed, we will investigate duplicate records separately.

Duplicate records can affect model training because the same observation may appear multiple times.

This section identifies exact duplicate rows without modifying the original dataset.

```
# Identify exact duplicate rows

duplicate_rows = final_df[
    final_df.duplicated(keep=False)
].copy()

print("Number of duplicate rows:", len(duplicate_rows))

if len(duplicate_rows) > 0:
    display(
        duplicate_rows.sort_values(
            by=["Brand", "model", "Year", "kmDriven", "AskPrice"]
        )
    )
else:
    print("No duplicate rows found.")
```